

ECI 39: Exploiting and securing communication on the Internet Day #1

Computer Networking and Security 101

Vladimír Veselý, veselyv@fit.vut.cz



27th July 2026

- Description

- *During the course, participants will learn about the basic principles of computer communication security. We will explain how authentication, confidentiality and integrity are achieved using various cryptographic techniques (e.g. MD5, SHA, AES and RSA). Next, participants will learn how to communicate securely on the web, send emails and connect to WiFi networks. Furthermore, we will demonstrate the security vulnerabilities in these scenarios using freely available hacking tools.*

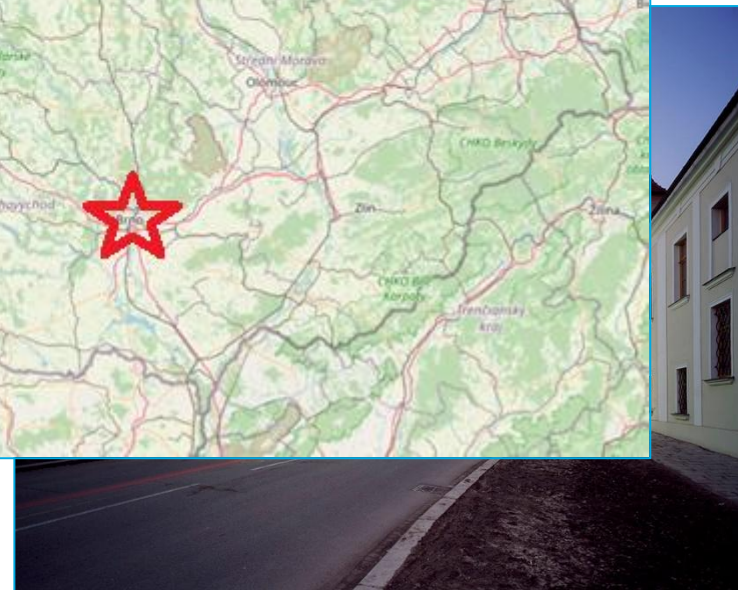
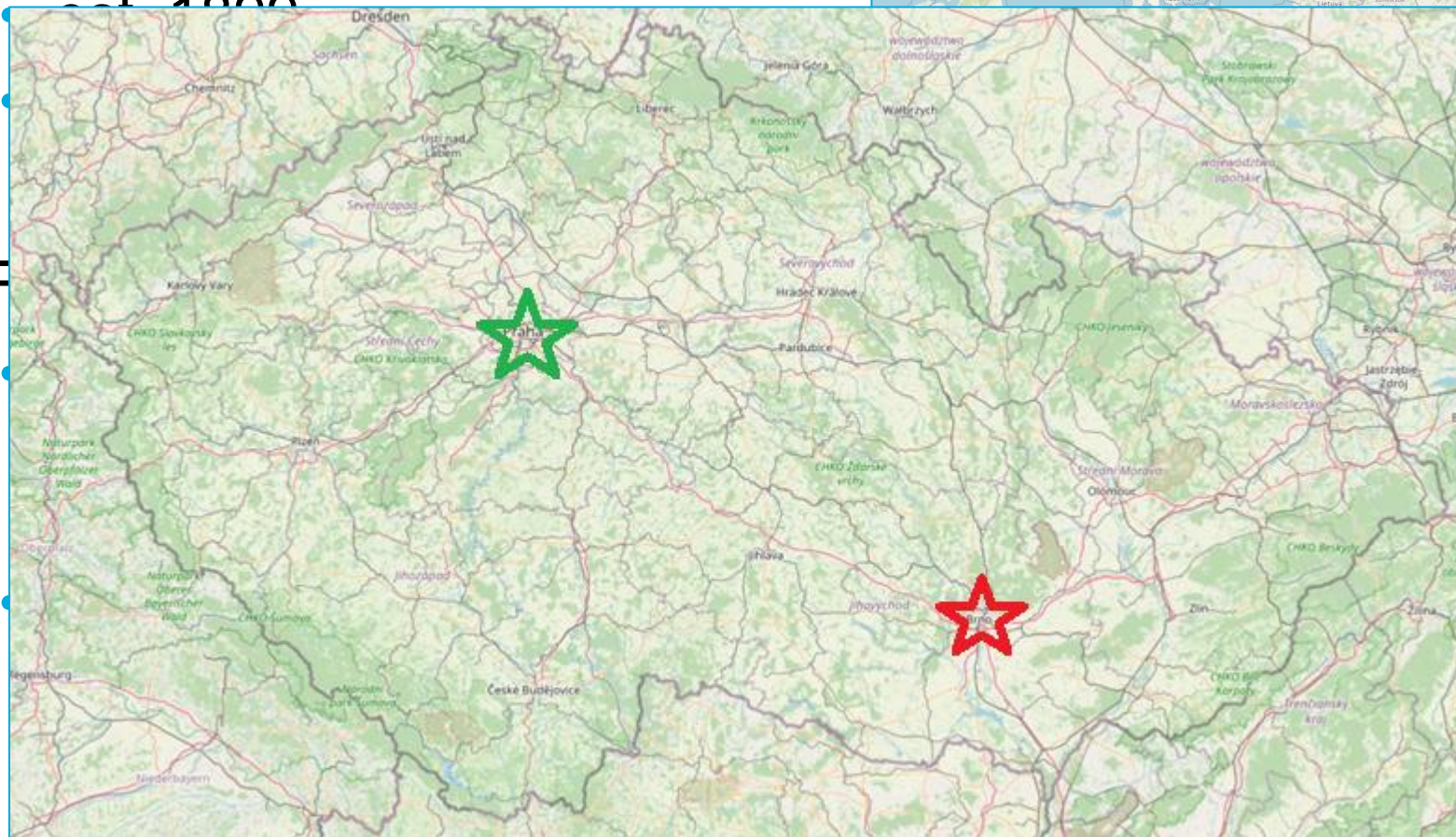
- Agenda

- Monday: Computer Networking and Security 101
- Tuesday: TLS/SSL and exploiting HTTP
- Wednesday: Exploiting DNS and SMTP
- Thursday: Exploiting DHCP and Wi-Fi
- Friday: Exploiting Wi-Fi and conducting MitM

- 25 000+ students

est. 1809

F



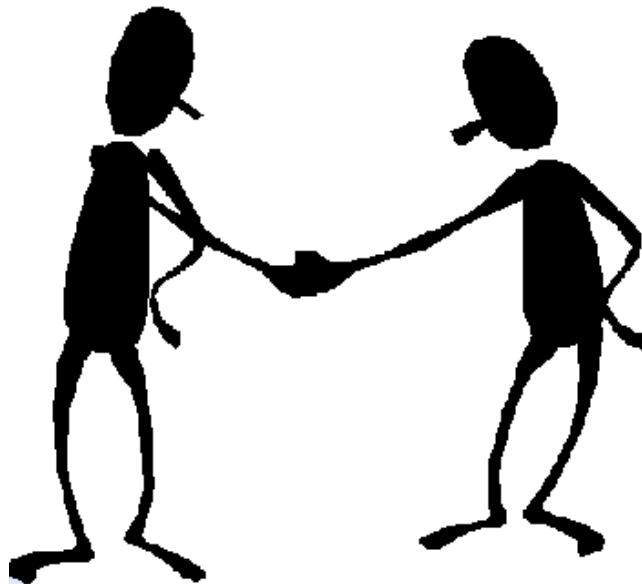
- 2 years in ANECT a.s.
 - networking consultant
- 17 years at Brno University of Technology
 - Ph.D. about alternative TCP/IP architectures
 - Lecturing bachelor and master classes
- 10 years at netsearch s.r.o.
 - RnD for law enforcement agencies

ANECT

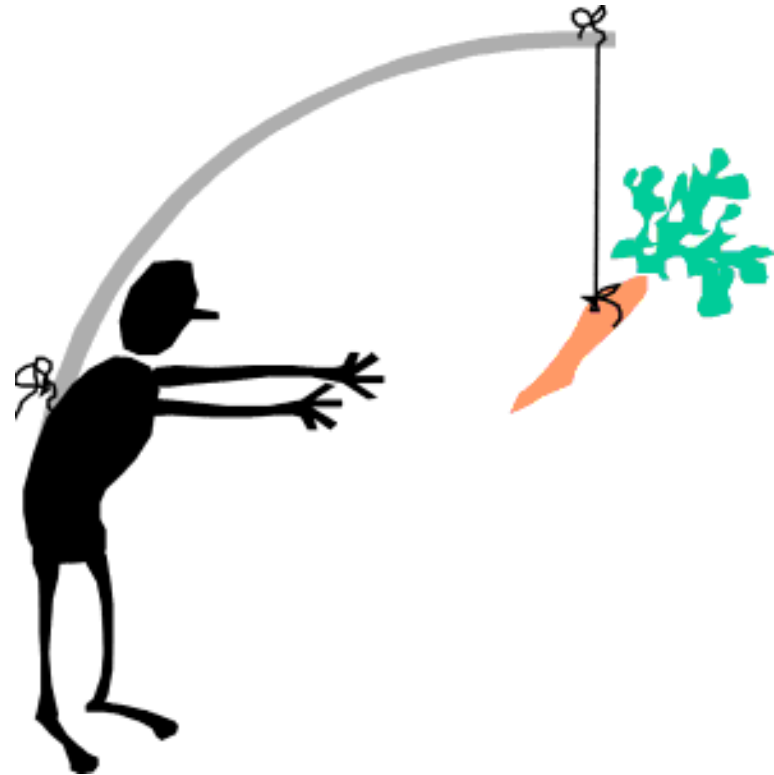


- Cisco Certified Instructor and Trainer





Who are you?

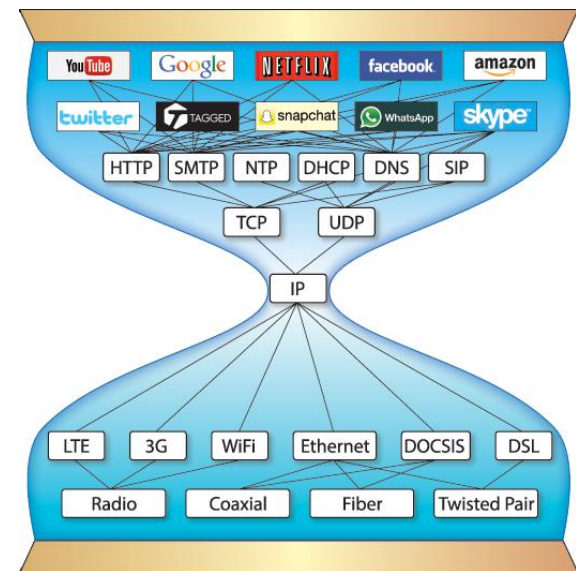


***What do you
expect from this
course?***

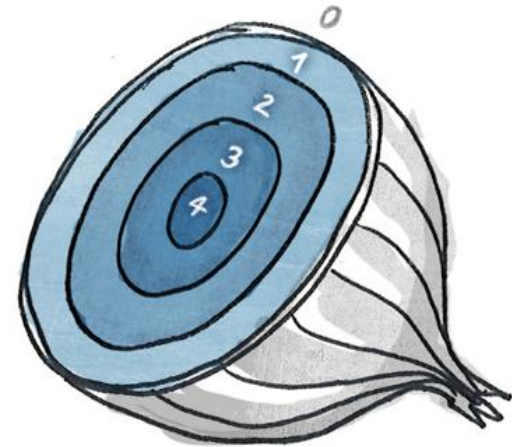
- <https://github.com/nesfit/ECI39>
- Slide decks
- Exercises

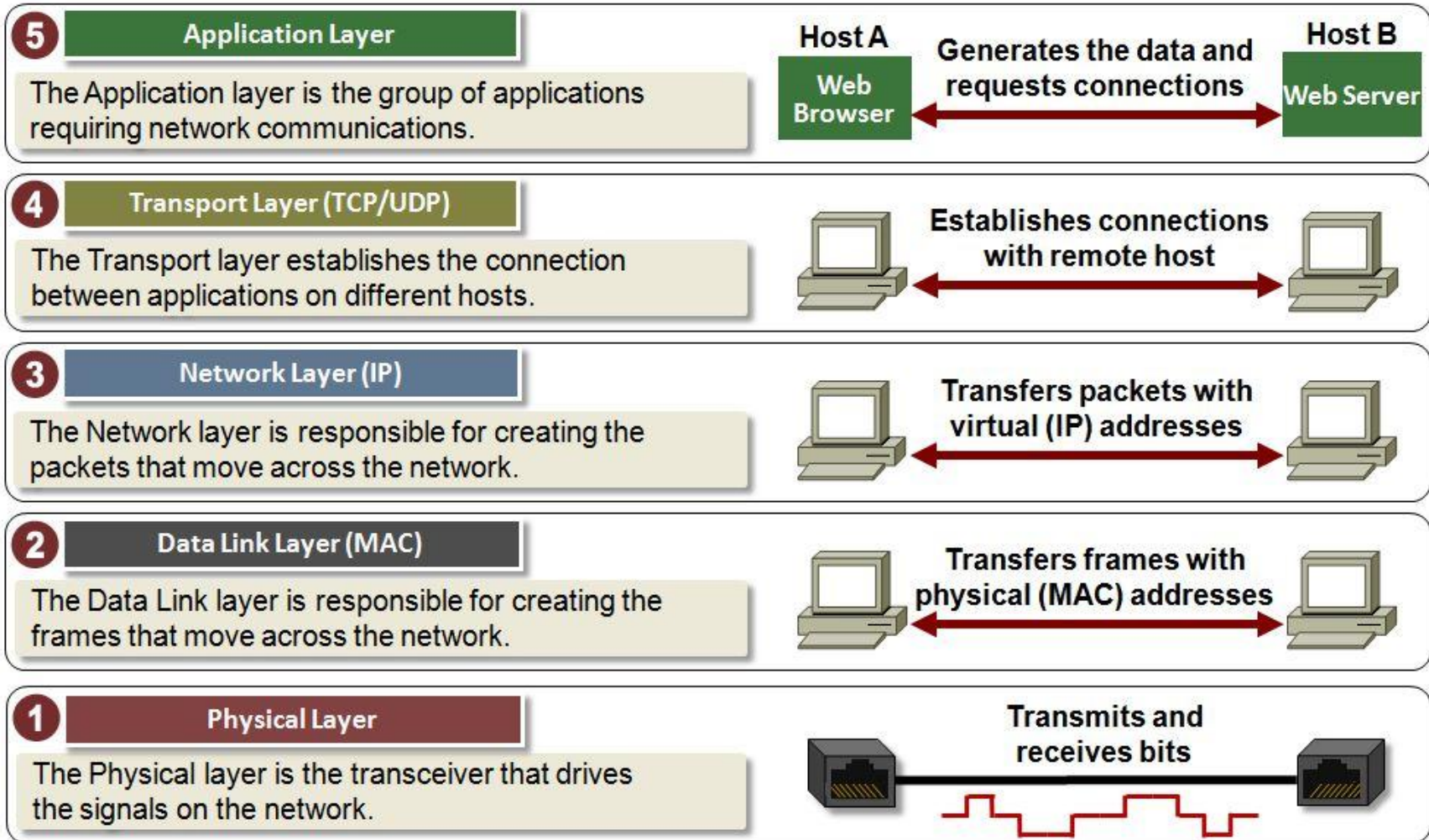
Networking 101

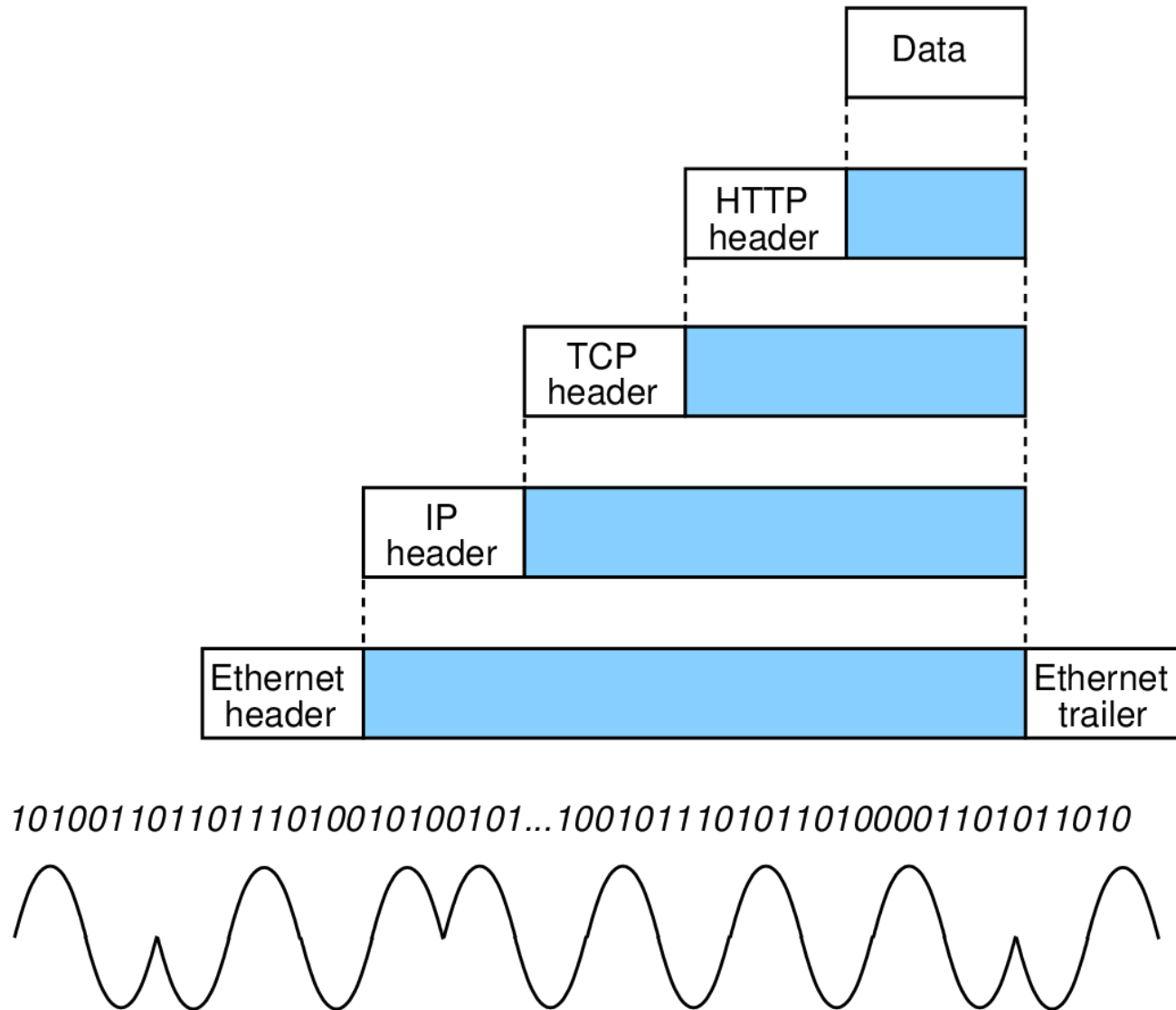
NETWORK STACK

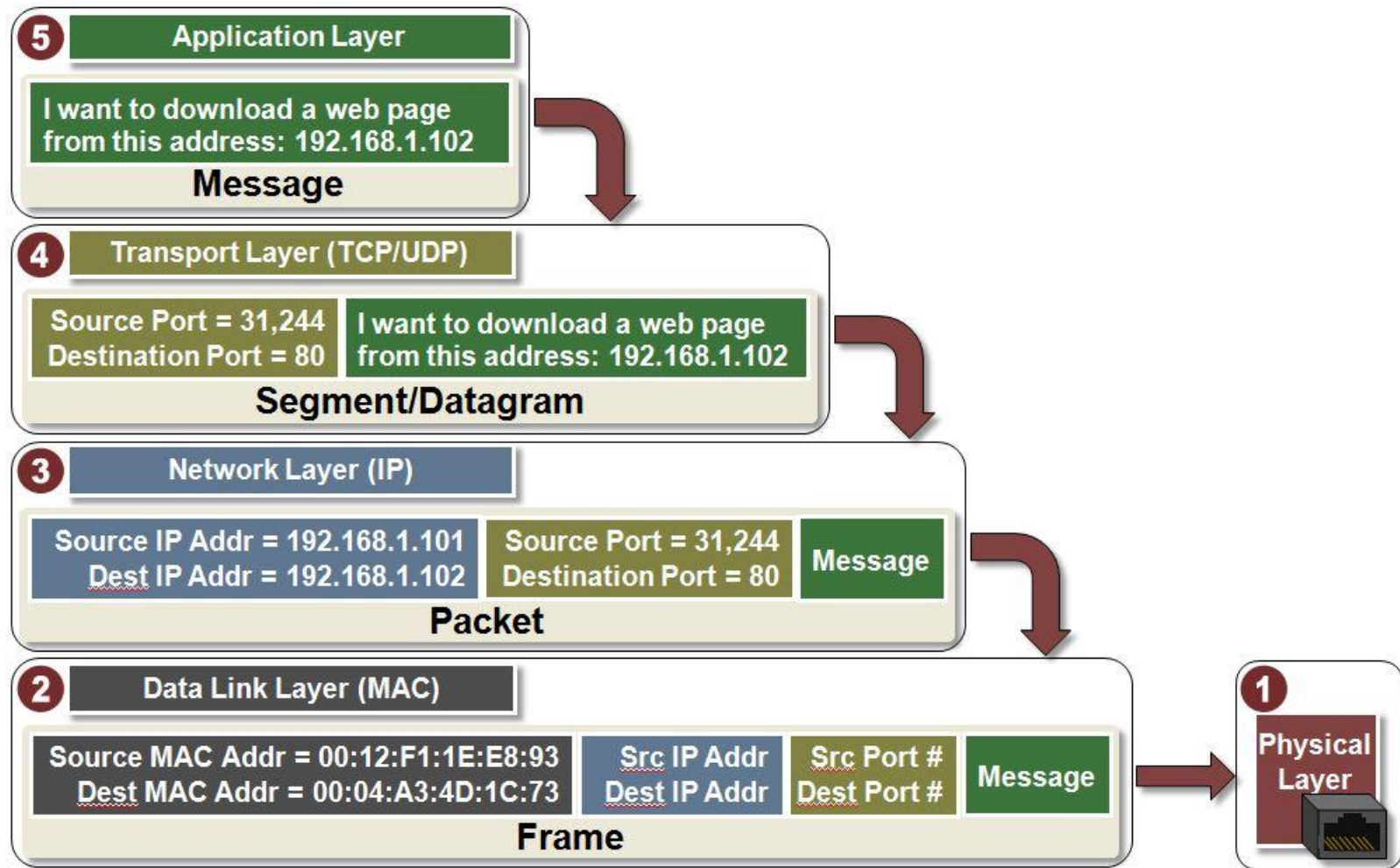


- Computer networks are complex systems
- Too many “parts”
 - End-systems
 - Routers and switches
 - Cable systems
 - Applications
 - Protocol
 - HW and SW
- Thus, division of scope into layers
- Referential layered models
 - ISO/OSI
 - TCP/IP
- Modular layers
 - Relationship between subsystems
 - Transparent change of layer due to the well-defined APIs
 - Extensible communication

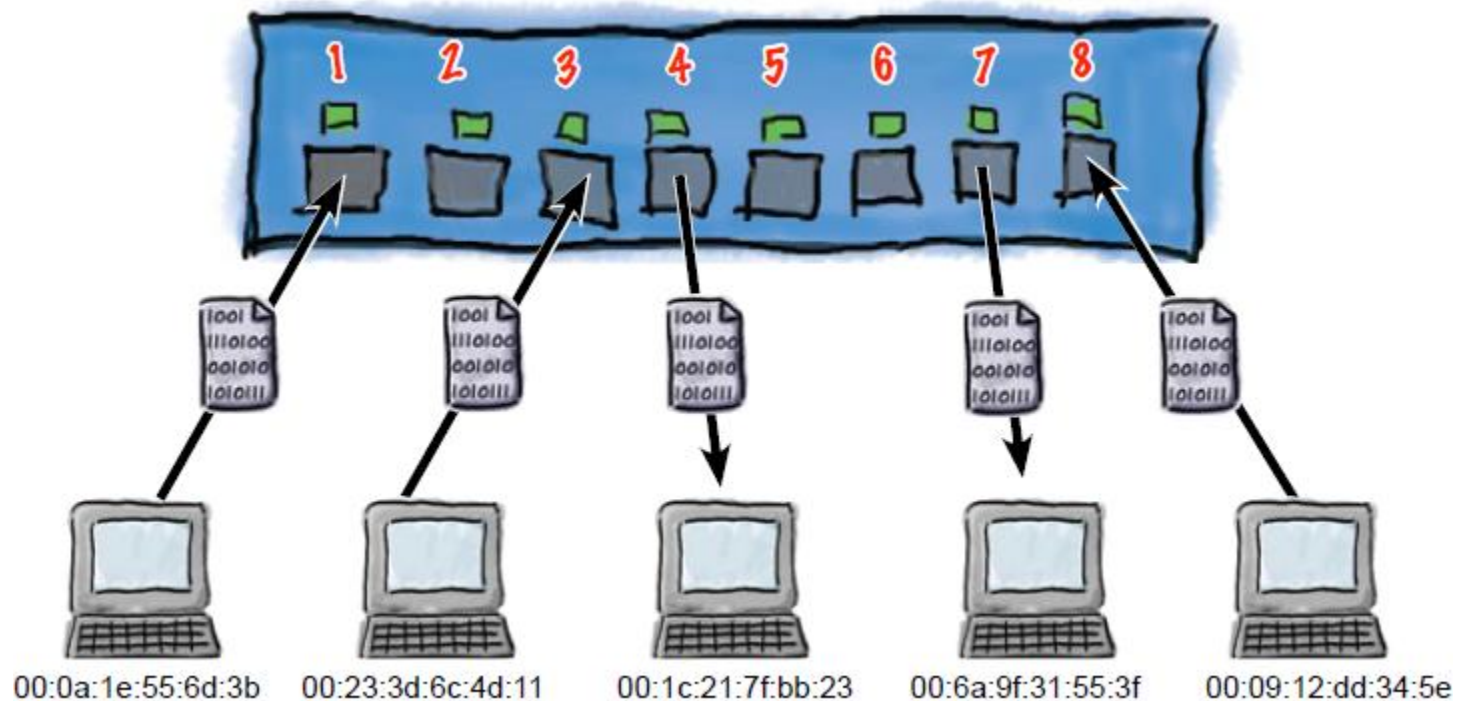




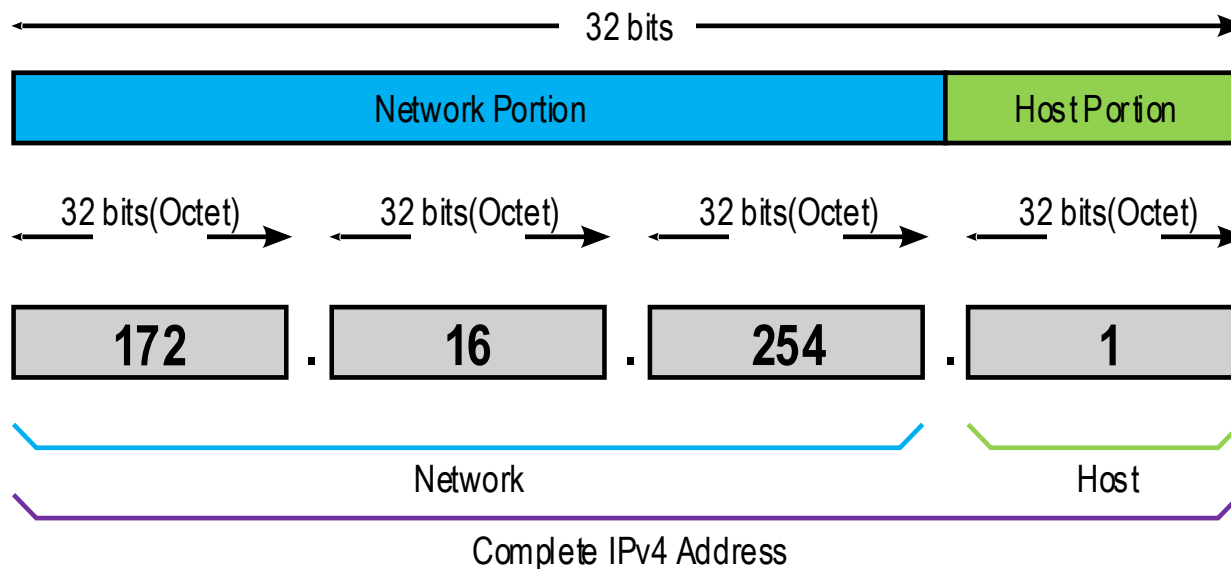




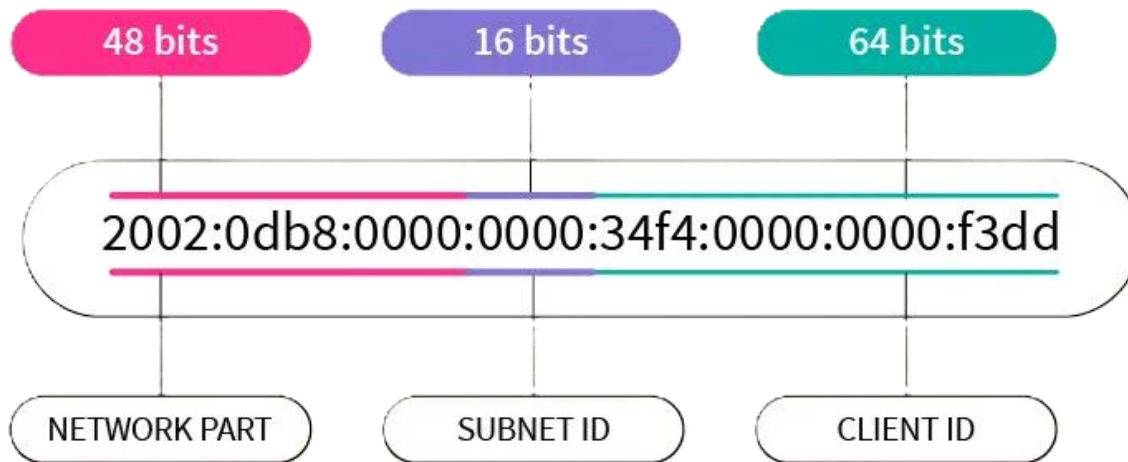
- 48-bits long **MAC address** burned to ROM of NIC
 - Each address should be unique (at least on the segment)
 - Flat address space assigned by IEEE



- 32b long L3 identifier
- Decimal groups divided by .
- Address consists of two parts:
network portion + host portion
 - E.g., 147.229.0.0 as network and 0.0.176.14 as host part



- 128 long L3 identifies
- Hexadecimal groups divided by :



- Leading 0 can be omitted, :: can replace longest group of zeros

`2031:0000:130f:0000:0000:09c0:876a:130b >>> 2031:0:130f::9c0:876a:130b`

`ff01:0:0:0:0:0:0:1 >>> ff01::1`

`0:0:0:0:0:0:0:1 >>> ::1`

`0:0:0:0:0:0:0:0 >>> ::`

- *Host knows*
 - IP address assigned by administrator
- *Host does not know*
 - MAC address assigned by manufacturer
- Glue between Layer 2 and Layer 3 addresses
 - **ARP** is for IPv4 to Ethernet MAC resolution
 - **ND** is for IPv6 to Ethernet MAC resolution
- When PDU are being encapsulated, host can't leave destination MAC address field blank.
- Each IP-to-MAC binding stored in local cache

PRACTICAL NETWORKING .NET



Server
10.0.0.33
0053.ffff.cccc

Router

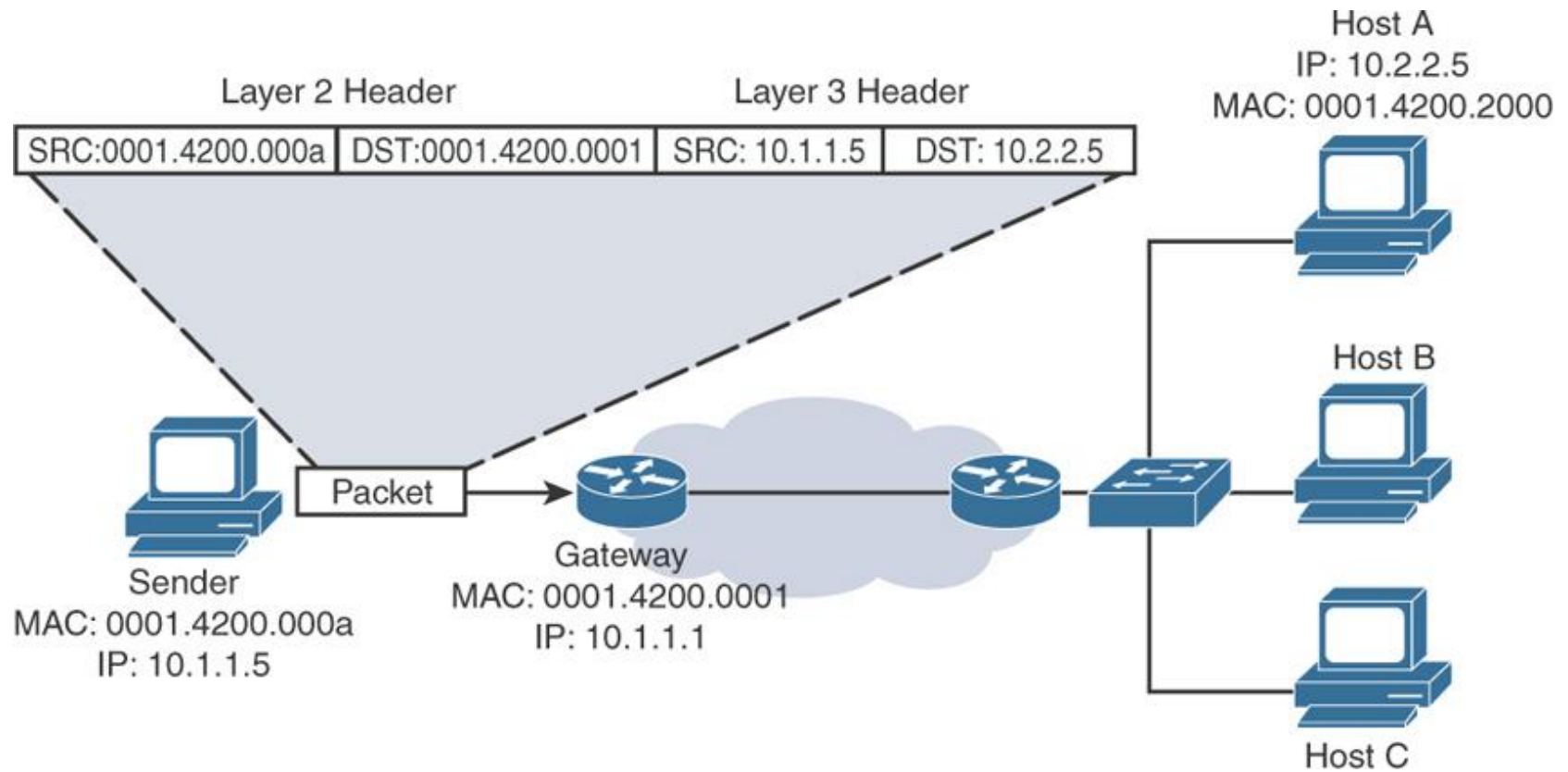
10.0.0.99
0053.ffff.9999

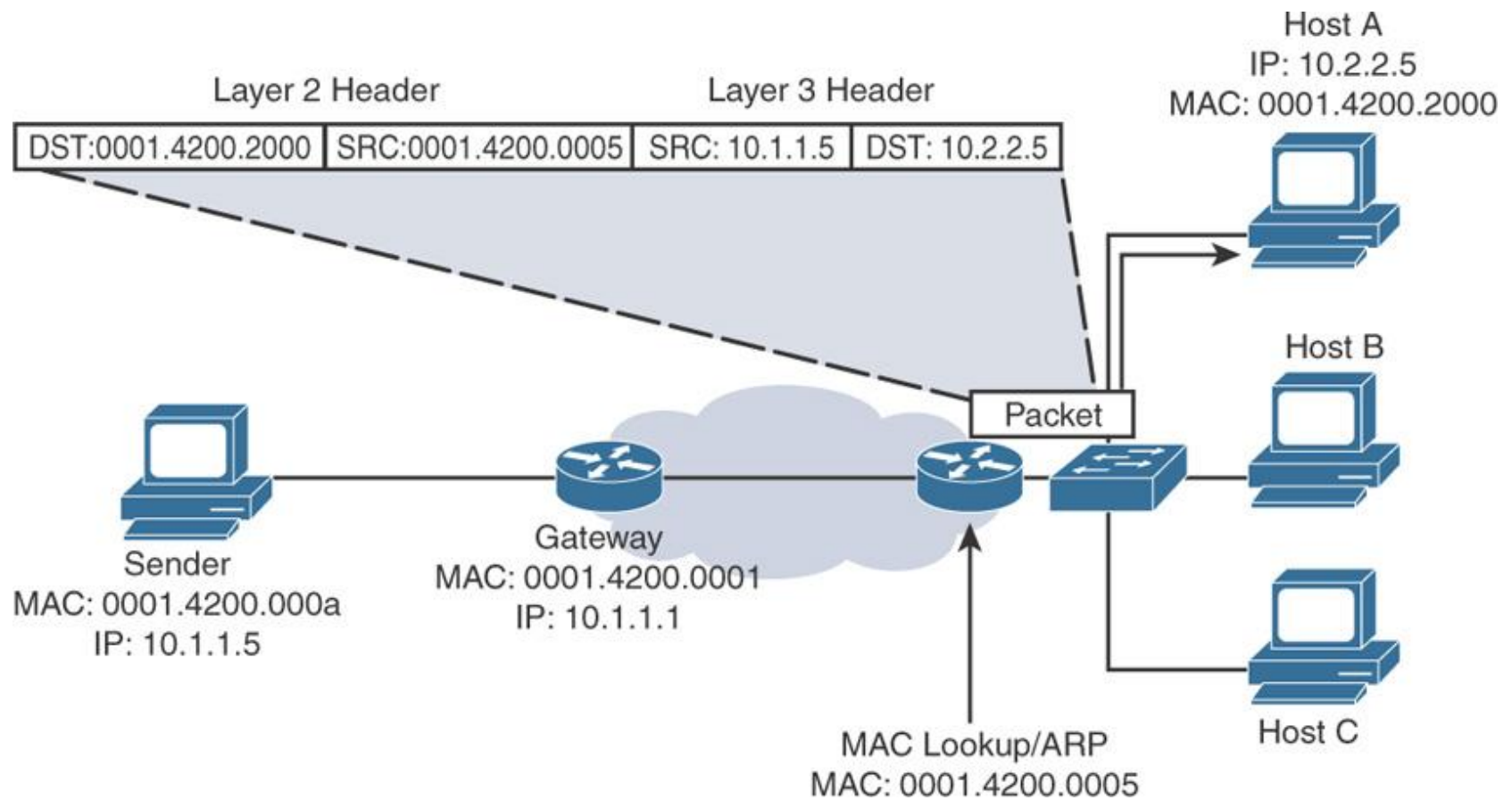


Host A
10.0.0.11
0053.ffff.aaaa



Host B
10.0.0.22
0053.ffff.bbbb





- L2:
 - WiFi: WPA123, WPS, protected management frames
 - Port security, DHCP spoofing/snooping, ARP MitM / Dynamic ARP inspection
- L3: GRE, IPsec, VPN – SSL (openvpn), wireguard
- L4: TLS/SSL, PKI
- L7:
 - DNS – DoH, DoT, DNSSEC
 - Email - DKIM, SPF, DMARC, PGP, S/MIME
 - Authentication: Radius, LDAP, OAuth, JWT

What tools are relevant

TOOLING



- LXC – mainly supported on Debian
- Docker
 - Established
 - DockerHub for container sharing
- OS vendor tools
 - RedHat Podman
- Orchestration tools
 - use containers to spin up groups of servers – Kubernetes, Docker Compose



LXC

LXC, Technically first implementation of Container



docker

Daemon process based Container Runtime Engine, Standardize container implementation



podman

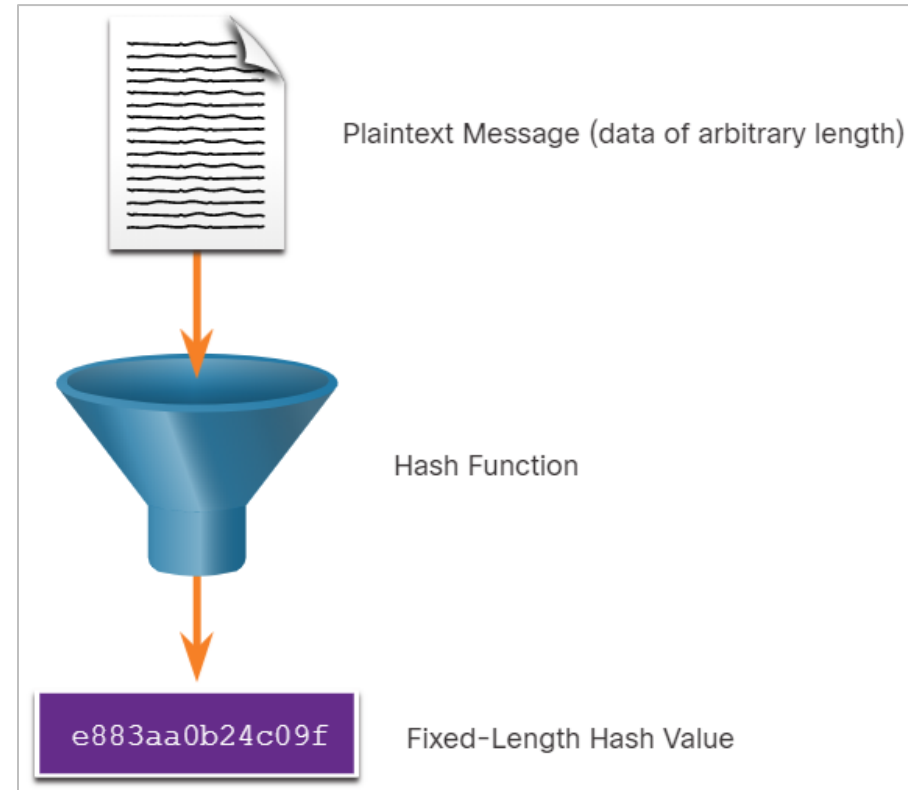
Opensource Solution for Containers, No Daemon Process, Security focused

Security 101

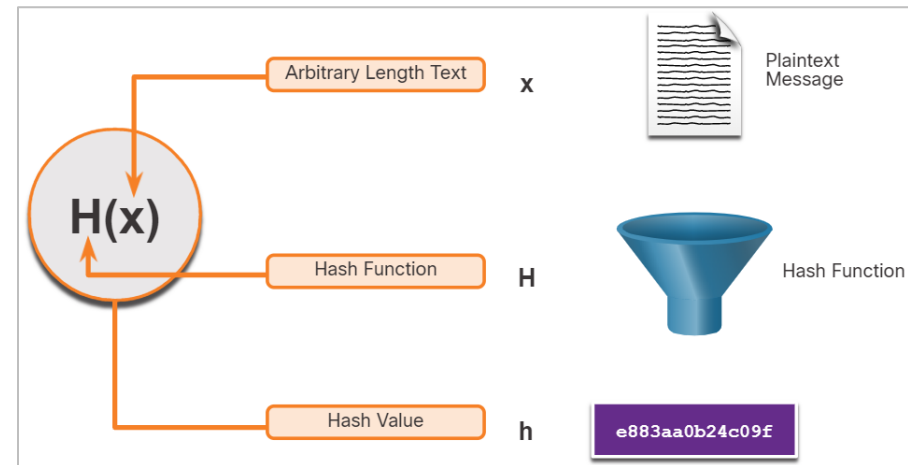
CRYPTOLOGY

- The four elements of securing communications are:
 - 1) Data **Integrity** - Guarantees that the message was not altered.
 - 2) Origin **Authenticity** - Guarantees that the message is not a forgery and it actually comes from whom it states.
 - 3) Data **Confidentiality** - Guarantees that only authorized users can read the message.
 - 4) Data **Non-Repudiation** - Guarantees that the sender cannot repudiate, or refute, the validity of a message sent.

- Hashes are used to verify and ensure data integrity
- Hashing is based on a one-way mathematical function that is relatively easy to compute, but significantly harder to reverse.
- A hash function takes a variable block of binary data, called the message, and produces a fixed-length, condensed representation, called the **hash**.
- The resulting hash is also sometimes called the message digest, digest, or digital fingerprint.
- **With hash functions, it is computationally infeasible for two different sets of data to come up with the same hash output.**
- Every time the data is changed or altered, the hash value also changes.



- Mathematically, the equation $h = H(x)$ is used to explain how a hash algorithm operates.
- As shown in the figure, a hash function H takes an input x and returns a fixed-size string hash value h .
- A cryptographic hash function should have the following properties:
 - The input can be any length.
 - The output has a fixed length.
 - $H(x)$ is relatively easy to compute for given x .
 - $H(x)$ is one way and not reversible.
 - $H(x)$ is collision free, meaning that two different input values will result in different hash values.



Algorithm	Description
MD5 (Message Digest Algorithm 5)	Produces a 128-bit hash value. It was commonly used for checksums and digital signatures. However, it is now considered cryptographically broken and unsuitable for further use due to collision vulnerabilities, where two different inputs can produce the same hash.
SHA-1 (Secure Hash Algorithm 1)	Produces a 160-bit hash value. Previously used in digital signatures and certificates. However, due to successful collision attacks, it is considered deprecated for security purposes and has been replaced by more secure SHA-2 algorithms.
SHA-256 (Secure Hash Algorithm 256-bit)	Part of the SHA-2 family, producing a 256-bit hash value. It is widely used in blockchain technology (e.g., Bitcoin), SSL/TLS certificates, and secure password hashing. SHA-256 offers strong collision resistance.
SHA-512 (Secure Hash Algorithm 512-bit)	Another member of the SHA-2 family, producing a 512-bit hash value. It is suitable for high-security applications such as digital certificates, digital signatures, and data integrity verification due to its longer output length.
RIPEMD-160 (RACE Integrity Primitives Evaluation Message Digest)	A cryptographic hash function producing a 160-bit hash value. It was designed to provide a secure alternative to MD4 and is used in some cryptocurrency protocols, such as Bitcoin for address generation.
BLAKE2	Produces a 256-bit or 512-bit hash value (depending on the variant: BLAKE2s for 256-bit and BLAKE2b for 512-bit). A fast and secure cryptographic hash function designed as an alternative to MD5 and SHA-2. It provides better security than MD5 and similar speed to MD5 but with fewer vulnerabilities. It is commonly used in cryptographic applications, digital signatures, and file integrity verification.
Whirlpool	A cryptographic hash function producing a 512-bit hash value, based on a modified version of the AES block cipher. It is recognized for its strength and is used in applications requiring data integrity, such as digital signatures and secure file storage.
CRC32 (Cyclic Redundancy Check 32-bit)	Produces a 32-bit hash value. A non-cryptographic hash function commonly used for error-checking in data transmission and storage. While it ensures data integrity, it is not suitable for cryptographic purposes due to its susceptibility to intentional collisions.

- `echo -n "test" | md5sum`
098f6bcd4621d373cade4e832627b4f6 -
- `echo -n "Test" | md5sum`
0cbc6611f5540bd0809a388dc95a615b -

- <https://emn178.github.io/online-tools/md5.html>

MD5

This MD5 online tool helps you calculate hashes from strings. You can input UTF-8, UTF-16, Hex, Base64,

Settings

Hash

☒ Auto Update
 ☐ Remember Input

Input Encoding

UTF-8

Output Encoding

Hex (Lower Case)

☐ Enable HMAC

Input

apple

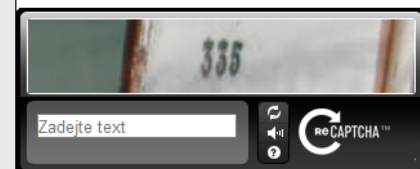
Output

1f3870be274f6c49b3e31a0c6728957f

- <https://crackstation.net/>

Enter up to 20 non-salted hashes, one per line:

1f3870be274f6c49b3e31a0c6728957f



Zadejte text

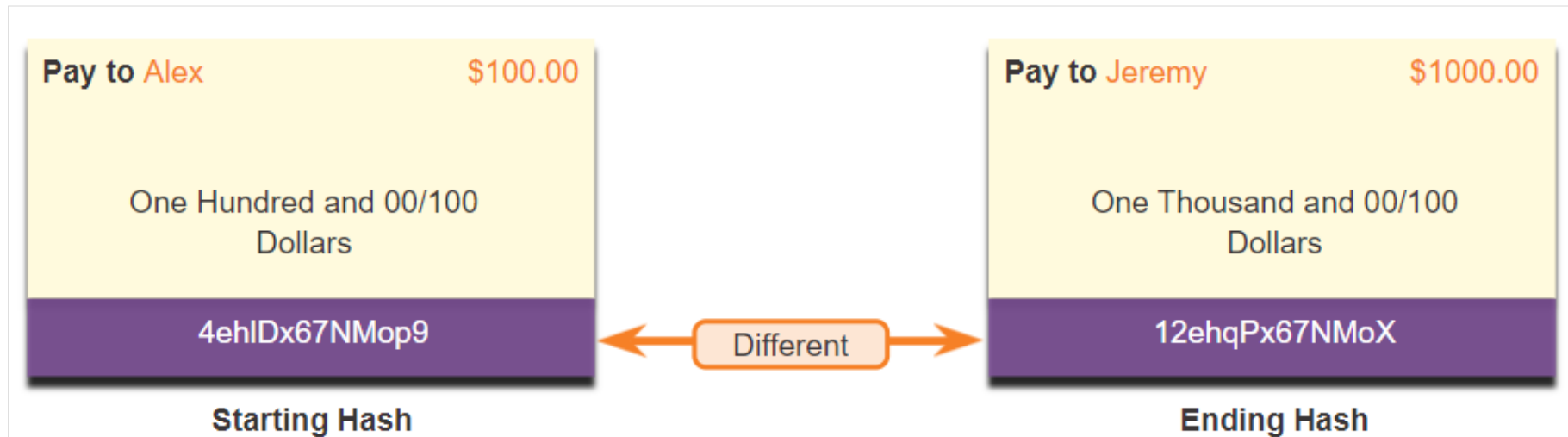


Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1 sha1_bin))

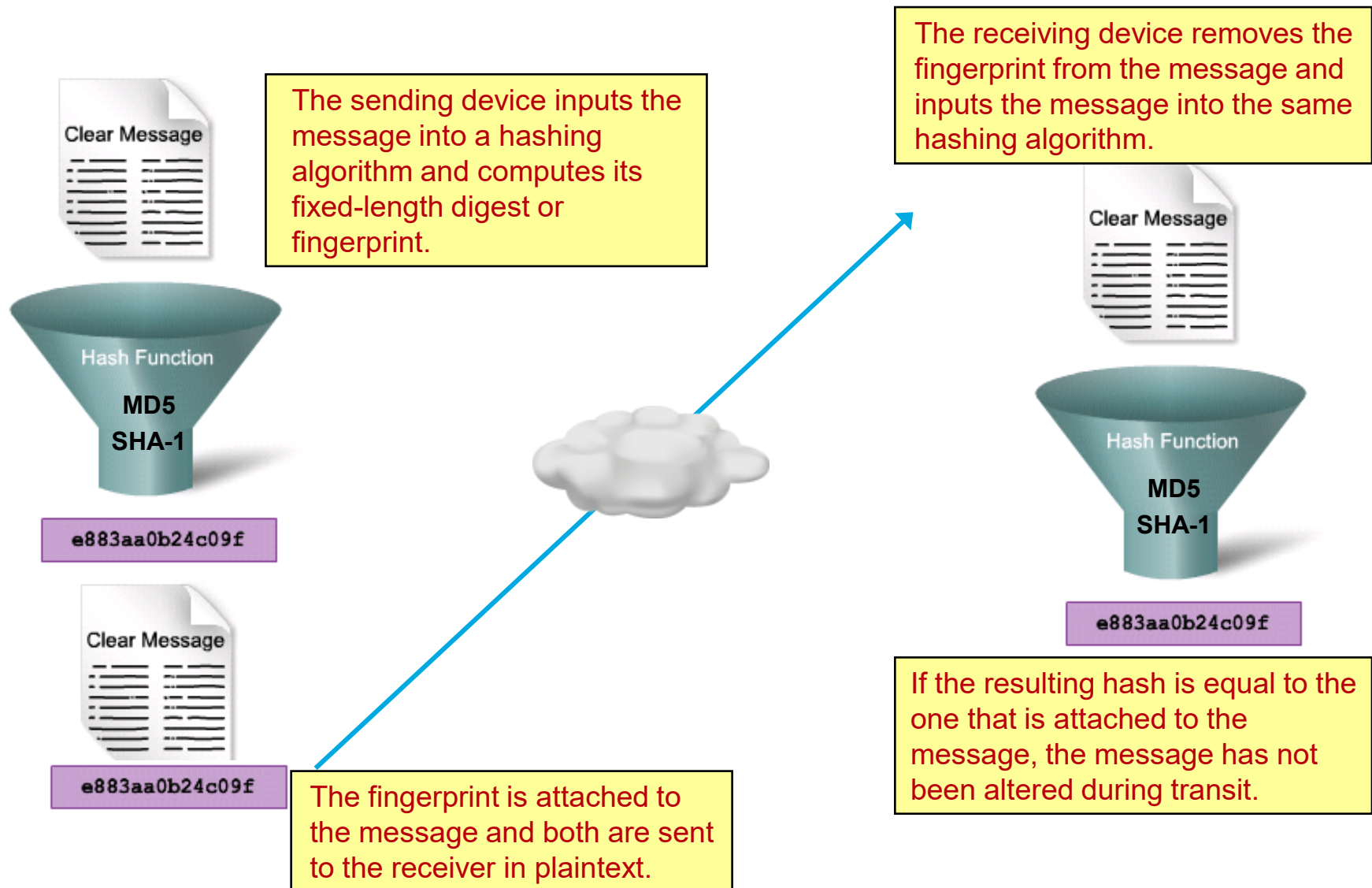
Hash	Type	Result
1f3870be274f6c49b3e31a0c6728957f	md5	apple

Color Codes: Exact match, Partial match, Not found.

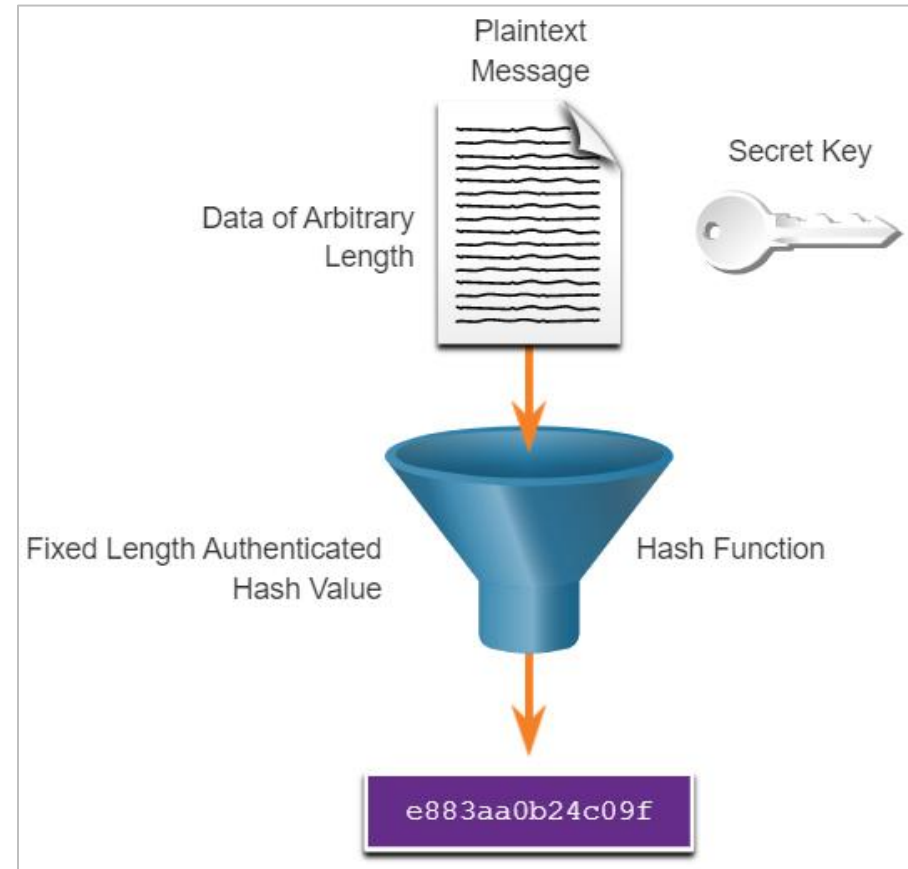


The sender wants to ensure that the message is not altered on its way to the receiver.

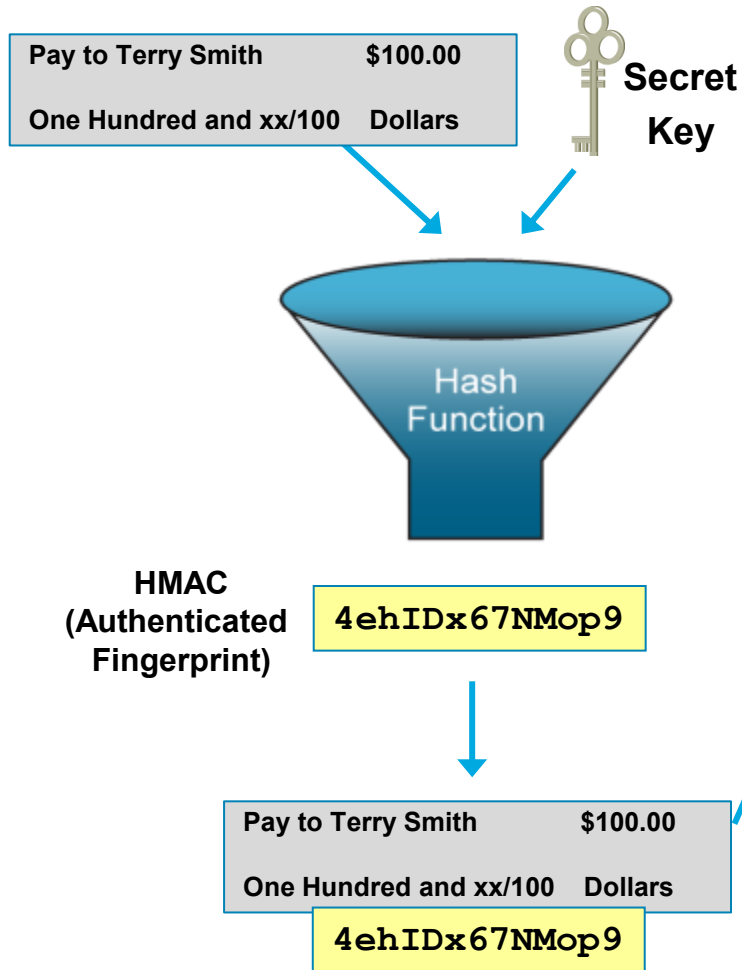
1. The sending device inputs the message into a hashing algorithm and computes its fixed-length hash of 4ehIDx67NMop9.
2. This hash is then attached to the message and sent to the receiver. Both the message and the hash are in plaintext.
3. The receiving device removes the hash from the message and inputs the message into the same hashing algorithm. If the computed hash is equal to the one that is attached to the message, the message has not been altered during transit. If the hashes are not equal, as shown in the figure, then the integrity of the message can no longer be trusted.



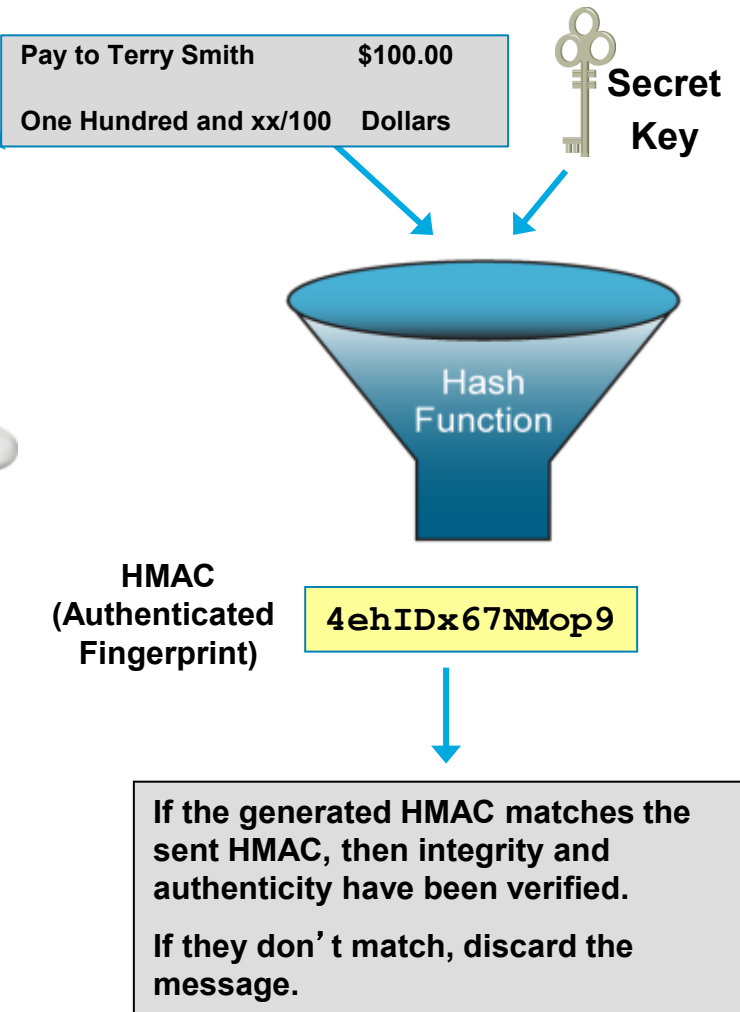
- **HMAC (or KMAC) is a message authentication code (MAC) that is calculated using a hash function and a secret key**
 - Cryptographic strength of the underlying hash function.
 - Size and quality of the key.
 - Size of the hash output length in bits.
- **Authenticity is guaranteed because only the sender and the receiver know the secret key**
 - Only they can compute the digest of an HMAC function.
 - This characteristic defeats man-in-the-middle attacks and provides authentication of the data origin.



Data



Received Data



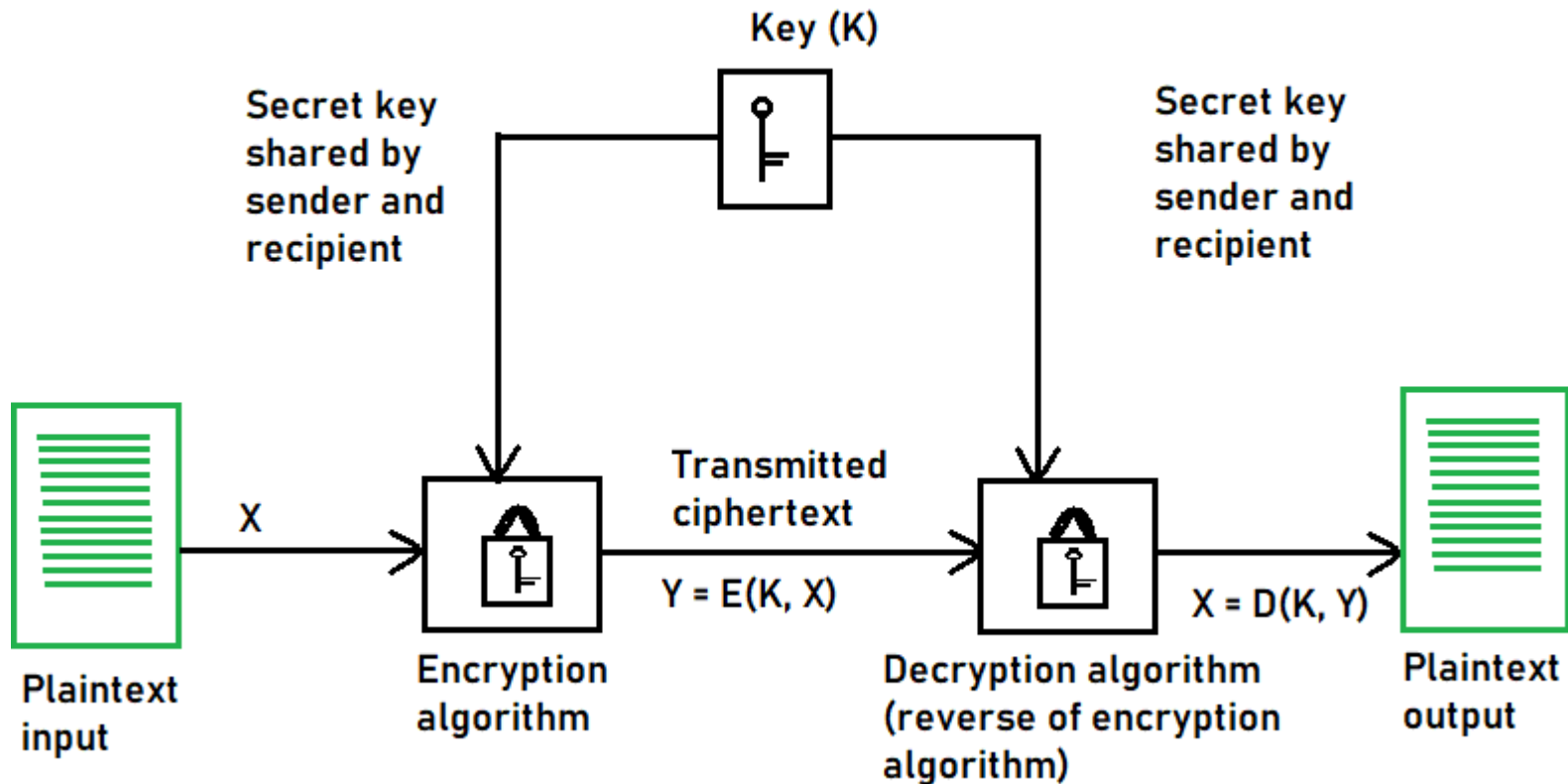
- **Key Generation:**
 - Caesar to choose the key of his cipher and the Sender/Receiver chose a shared secret key for the Vigenère cipher.
 - Modern cryptographic system key generation is usually automated.
- **Key Verification:**
 - Almost all cryptographic algorithms have some weak keys that should not be used (e.g., Caesar cipher ROT 0 or ROT 25).
 - With the help of key verification procedures, these keys can be regenerated if they occur.
- **Key Storage:**
 - Modern cryptographic system store keys in memory.
- **Key Exchange:**
 - Key management procedures should provide a secure key exchange mechanism over an untrusted medium.
- **Key Revocation and Destruction:**
 - Revocation notifies all interested parties that a certain key has been compromised and should no longer be used.
 - Destruction erases old keys in a manner that prevents malicious attackers from recovering them.

- Adding one bit to a key doubles the keyspace.
- For each bit added to the DES key, the attacker would require twice the amount of time to search the keyspace.
- Longer keys are more secure but are also more resource intensive and can affect throughput.



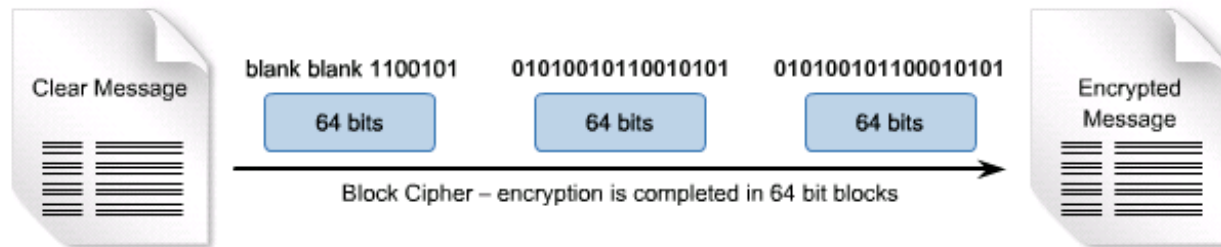
DES Key Length	Keyspace	# of Possible Keys
56 bit	2^{56}	72,000,000,000,000,000
57 bit	2^{57}	144,000,000,000,000,000
58 bit	2^{58}	288,000,000,000,000,000
59 bit	2^{59}	576,000,000,000,000,000

- Symmetric encryption algorithms, also called shared secret-key algorithms, **use the same pre-shared secret key to encrypt and decrypt data.**
 - The **pre-shared key is known by the sender and receiver** before any encrypted communications begins.
- Because both parties are guarding a shared secret, the encryption algorithms used can have shorter key lengths.
 - Shorter key lengths mean faster execution.
- For this reason symmetric algorithms are generally much less computationally intensive than asymmetric algorithms.

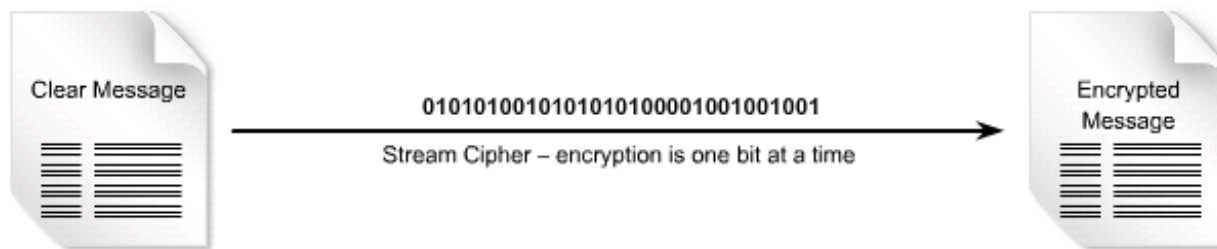


- <https://www.geeksforgeeks.org/symmetric-key-cryptography/>

- **Block ciphers** transform a fixed-length block of plaintext into a common block of ciphertext of 64 or 128 bits.
 - Block size refers to how much data is encrypted at any one time.



- **Stream ciphers** encrypt plaintext one byte or one bit at a time.
 - Think of it like a block cipher with a block size of one bit.
 - The Vigenère cipher is an example of a stream cipher.
 - Can be much faster than block ciphers, and generally do not increase the message size.



Algorithm	Description, Key Sizes, and Author
AES (Advanced Encryption Standard)	Developed by Vincent Rijmen and Joan Daemen. A widely-used symmetric encryption standard adopted by the U.S. government. Uses substitution-permutation network (SPN) and supports key sizes of 128, 192, and 256 bits. Commonly used in TLS/SSL, VPNs, and disk encryption.
DES (Data Encryption Standard)	Developed by IBM, with input from the NSA. A symmetric block cipher from the 1970s with a 56-bit key. Uses 16 rounds of Feistel structure but is now obsolete due to brute-force vulnerabilities.
3DES (Triple DES)	Developed by IBM. Improves DES by applying the algorithm three times to each data block. Uses two or three 56-bit keys (112 or 168 bits total). Common in legacy systems.
Blowfish	Developed by Bruce Schneier. A symmetric block cipher with a 64-bit block size and variable key sizes (32 to 448 bits). Known for speed and used in password hashing and file encryption.
Twofish	Developed by Bruce Schneier and his team. A successor to Blowfish with a 128-bit block size and keys up to 256 bits. Known for flexibility and used in TrueCrypt and other encryption software.
RC4 (Rivest Cipher 4)	Developed by Ron Rivest. A stream cipher known for simplicity and speed, using key sizes from 40 to 2048 bits. Once popular in WEP and SSL/TLS but now considered insecure.
ChaCha20	Developed by Daniel J. Bernstein. A modern stream cipher providing high security and performance. Uses a 256-bit key and is based on a variant of the Salsa20 algorithm. Commonly used in TLS 1.3, OpenSSH, and mobile VPNs.
IDEA (International Data Encryption Algorithm)	Developed by James Massey and Xuejia Lai. A symmetric block cipher known for use in PGP (Pretty Good Privacy). Uses a 64-bit block size and a 128-bit key. IDEA is secure but patented, limiting its widespread use.
GOST (Gosudarstvennyy Standard)	Developed by the Soviet Academy of Sciences. A symmetric cipher with a 256-bit key and 32 rounds of Feistel structure. Part of Russian national standards, used for secure communications.
Camellia	Developed by Mitsubishi Electric and NTT. A symmetric block cipher similar in design to AES. Supports key sizes of 128, 192, and 256 bits. Optimized for software and hardware implementation, commonly used in SSL/TLS and mobile security.

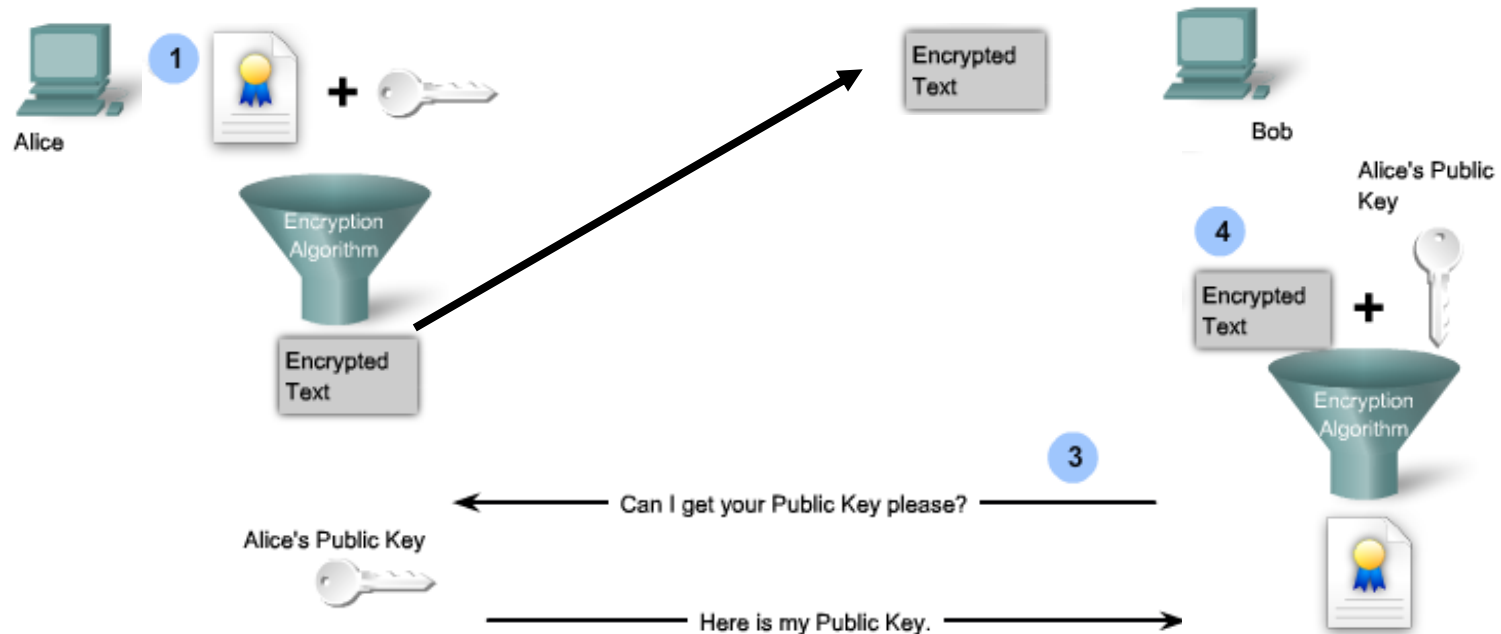
- **Encrypt (AES-256-CBC)**
 - `openssl enc -aes-256-cbc
-salt -pbkdf2 -iter 200000
-in plaintext.txt -out encrypted.bin`
- **Decrypt**
 - `openssl enc -d -aes-256-cbc
-pbkdf2 -iter 200000
-in encrypted.bin -out decrypted.txt`

- Asymmetric encryption algorithms, also called public/private key algorithms, **use two different keys to encrypt and decrypt data**
- Secure messages can be exchanged without having to have a pre-shared key.
- Because both parties do not have a shared secret, very long key lengths must be used to thwart attackers.
 - These algorithms are resource intensive and slower to execute.
- *In practice, asymmetric algorithms are typically 100 to 1,000 times slower than symmetric algorithms.*



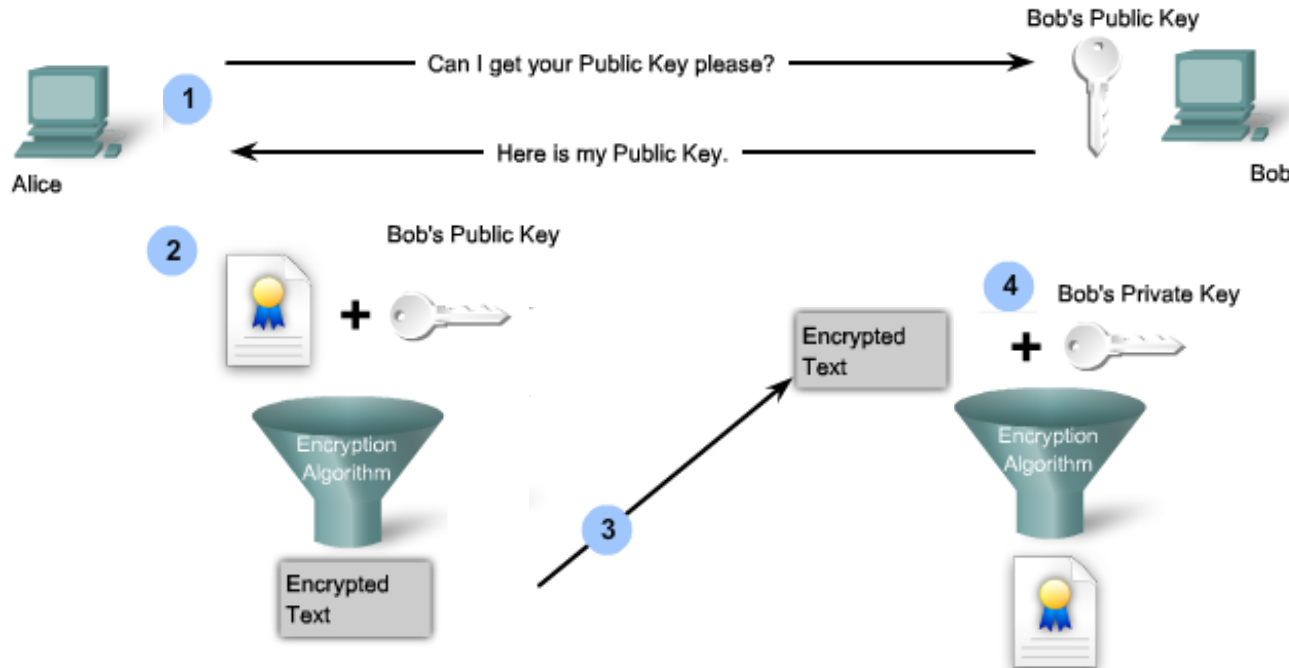
- <https://www.geeksforgeeks.org/asymmetric-key-cryptography/>

Private Key (Encrypt) + Public Key (Decrypt) = Authentication

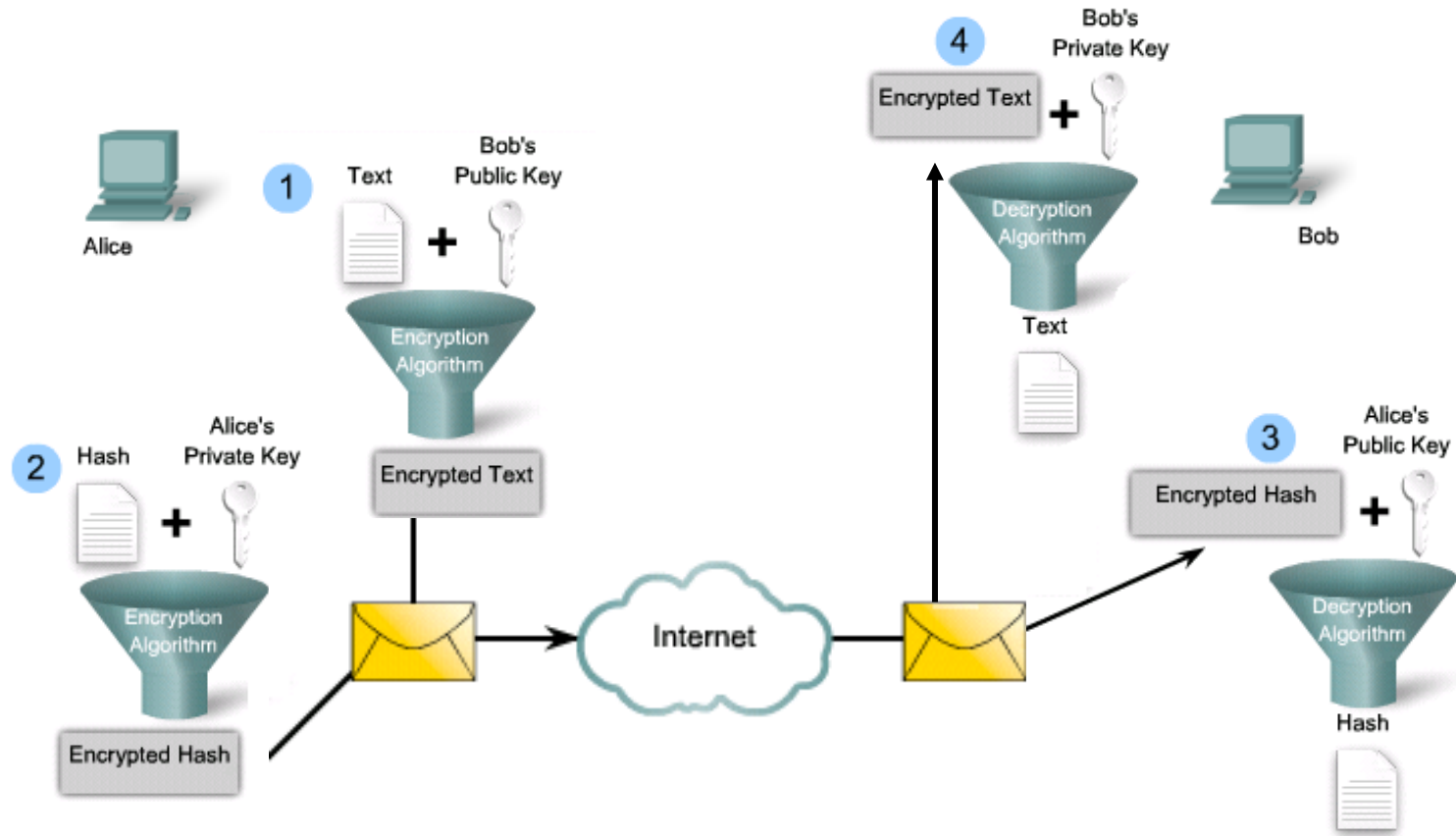


- 1) Alice will encrypt the message with her own private key.
- 2) Alice sends the ciphertext to Bob.
- 3) Bob requests Alice's public key to authenticate the message.
- 4) To verify that the message is from Alice, Bob uses Alice's public key to decrypt the message. If the message is readable, the authenticity of Alice's message is verified.

Public Key (Encrypt) + Private Key (Decrypt) = Confidentiality



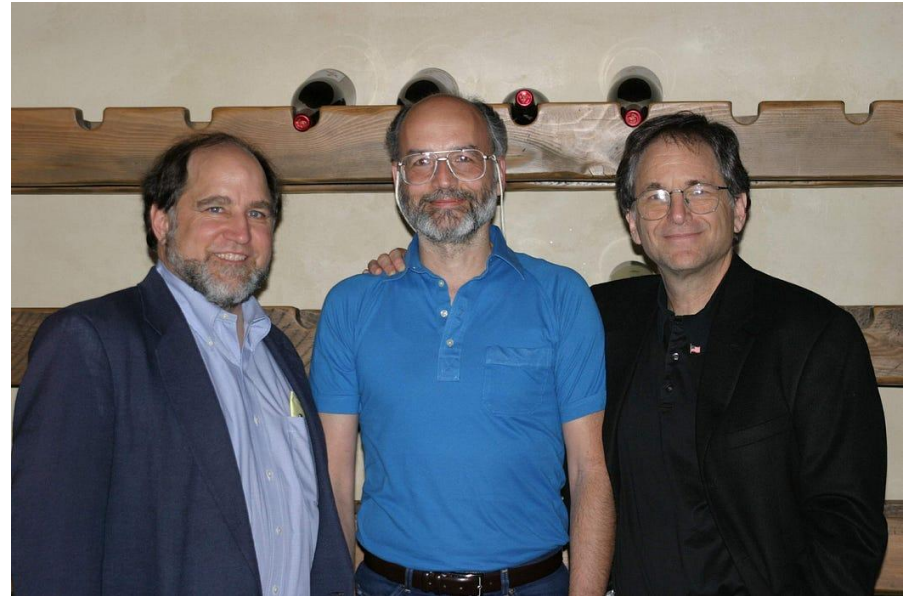
- 1) Alice asks Bob for his public key.
- 2) Alice uses Bob's public key to encrypt the message using a pre-agreed algorithm.
- 3) Alice sends Bob the cipher-text.
- 4) Bob uses his private key to decrypt the message.



- 1) Alice encrypts the message using Bob's public key.
- 2) Alice encrypts the HMAC with her own private key.
- 3) Bob uses Alice's public key to authenticate the HMAC message.
- 4) Bob uses his private key to decrypt the cipher-text.

Algorithm	Description, Key Sizes, and Author
RSA (Rivest-Shamir-Adleman)	Developed by Ron Rivest, Adi Shamir, and Leonard Adleman. RSA is one of the most widely used public-key cryptosystems. It uses key sizes from 1024 to 4096 bits and is based on the difficulty of factoring large integers. Commonly used for digital signatures and secure data transmission.
Diffie-Hellman Key Exchange	Developed by Whitfield Diffie and Martin Hellman. Diffie-Hellman is a method for securely exchanging cryptographic keys over a public channel. It is the basis for many secure protocols but does not provide encryption or digital signatures alone.
ECC (Elliptic Curve Cryptography)	Developed by Neal Koblitz and Victor S. Miller. ECC provides the same level of security as RSA with smaller key sizes (256-bit ECC is equivalent to 3072-bit RSA). Widely used in modern cryptographic systems, including TLS, Bitcoin, and mobile security.
ElGamal	Developed by Taher Elgamal. A public-key cryptosystem based on the Diffie-Hellman key exchange. Commonly used in digital signature schemes and hybrid encryption protocols. Uses key sizes of 1024 bits and above for security.
DSA (Digital Signature Algorithm)	Developed by the U.S. National Security Agency (NSA). DSA is a Federal Information Processing Standard (FIPS) for digital signatures. It is based on modular exponentiation and is commonly used for authentication and integrity verification. Key sizes range from 1024 to 3072 bits.
Ed25519	Developed by Daniel J. Bernstein. Ed25519 is a high-performance, secure elliptic curve algorithm used for digital signatures. It is based on the Edwards curve and provides strong security with a compact 256-bit key size. Popular in modern security protocols such as OpenSSH and TLS.
Paillier Cryptosystem	Developed by Pascal Paillier. A probabilistic asymmetric algorithm known for its homomorphic properties. Paillier is used in privacy-preserving computations and secure voting systems. Typical key sizes range from 1024 to 4096 bits.
McEliece Cryptosystem	Developed by Robert McEliece. A code-based public-key cryptosystem known for its resistance to quantum attacks. It uses large key sizes (typically 512 KB to 1 MB) but is highly secure against current and post-quantum attacks.
NTRUEncrypt	Developed by Jeffrey Hoffstein, Jill Pipher, and Joseph H. Silverman. NTRU is a lattice-based asymmetric encryption algorithm designed for high security and efficiency. It is considered a strong candidate for post-quantum cryptography. Key sizes vary but are generally compact compared to other lattice-based systems.

- RSA is an encryption/signing algorithm
 - Method developed by Ron Rivest, Adi Shamir and Len Adleman based on integer factorization in 1977
- RSA allows directly encrypt data with someone's public key, or sign data with private key



- 1) Select two prime numbers p and q where $p \neq q$
- 2) Calculate $n = p * q$
- 3) Calculate $\Phi(n) = (p - 1) * (q - 1)$
- 4) Select e such that, e is relatively prime to $\Phi(n)$, i.e. $(e, \Phi(n)) = 1$ and $1 < e < \Phi(n)$
- 5) Calculate $d = e^{-1} \bmod \Phi(n)$ or $e * d = 1 \bmod \Phi(n)$
- 6) Public key = $\{e, n\}$,
private key = $\{d, n\}$.
- 7) Find out cipher text using the formula, $C = P^e \bmod n$
- 8) Plain text P can be obtain using the given formula, $P = C^d \bmod n$.

Exercise - 1

Question: P and Q are two prime numbers. P=7, and Q=17. Take public key E=5. If plain text value is 6, then what will be cipher text value according to RSA algorithm? Again calculate plain text value from cipher text.

Solution:

1. Two prime numbers P=7, Q=17
2. $n = P * Q = 17 * 7 = 119$ **n = 119**
3. $\Phi(n) = (P-1) * (Q-1) = (17-1) * (7-1) = 16 * 6 = 96$ **$\Phi(n) = 96$**
4. Public key E = 5. **E = 5**
5. Calculate d = 77. $d = ((\Phi(n) * i) + 1) / e$ **d = 77**
 $d = ((96*1)+1) / 5 = 19.4$
 $d = ((96*2)+1) / 5 = 38.6$
 $d = ((96*3)+1) / 5 = 57.8$
 $d = ((96*4)+1) / 5 = 77$ (Stop finding d because getting integer value)
6. Public key = $\{e, n\} = \{5, 119\}$, private key = $\{d, n\} = \{77, 119\}$.
7. Plain text PT = 6, $CT = PT^E \bmod n = 6^5 \bmod 119 = 41$. **Cipher Text = 41**
8. Cipher text CT = 41, $PT = CT^d \bmod n = 41^{77} \bmod 119 = 6$. **Plain Text = 6**

Modulus Size (bits)	Approx. Decimal Digits	Symmetric equivalent strength (bits)	Status / Typical Use
512	~154	~50	Broken — factored in hours on modern hardware; never use
768	~232	~65	Broken — factored publicly in 2009; unsafe
1024	~309	~80	Deprecated — considered breakable by well-resourced attackers; phased out by major CAs since ~2013
1536	~463	~90	Rarely used; transitional size
2048	~617	~112	Current baseline minimum — widely used for TLS certs, SSH keys, PGP
3072	~925	~128	Matches AES-128 security; recommended for longer-term protection
4096	~1234	~140-150	Common "extra safe" choice; used for long-lived root CAs, high-security keys
7680	~2311	~192	Matches AES-192; rarely used due to performance cost
15360	~4622	~256	Matches AES-256; essentially never used in practice (too slow)

- **Modulus size** = $n = p \times q$, where p and q are two large primes each roughly half the modulus's bit length.
- NIST and most industry standards recommend 2048-bit as the minimum, with 3072-bit or higher for data that needs to stay secure beyond 2030.
- Larger moduli mean slower encryption/signing and larger ciphertexts/signatures
 - there's a real performance tradeoff
 - which is why ECC (elliptic curve) keys are increasingly preferred for equivalent security at much smaller key sizes (e.g., a 256-bit ECC key $\approx$ 3072-bit RSA)

1) Generate RSA private key + public key

- Private key (2048-bit)
 - `openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out private.pem`
- Public key (extracted from private)
 - `openssl pkey -in private.pem -pubout -out public.pem`

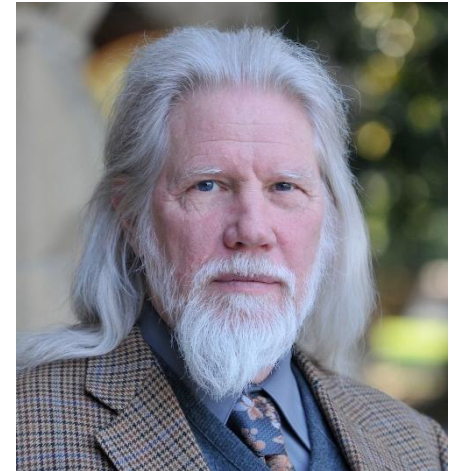
2) Encrypt with the public key

- `echo „I love ECI39 ;-)" > msg.txt`
- `openssl pkeyutl -encrypt -pubin -inkey public.pem -in msg.txt -out msg.enc -pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256`

3) Decrypt with the private key

- `openssl pkeyutl -decrypt -inkey private.pem -in msg.enc -out msg.dec -pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256`

- Imagine Alice and Bob wanting to communicate securely over an insecure channel.
 - They need a way to agree on a secret key without actually exchanging it.
 - Method based on discrete logarithm problem developed by Whitfield Diffie and Martin Hellman in 1976.
- DH is key exchange protocol
- Diffie-Hellman allows two parties to establish a shared secret key without directly exchanging the key itself.



1) Public Parameters (agreed publicly)

- Large prime number p
- Base (generator) g

2) Private/Public Key Generation

- Alice chooses a private number a , computes public key: $A = g^a \bmod p$.
- Bob chooses a private number b , computes public key $B = g^b \bmod p$.

3) Exchange Public Keys

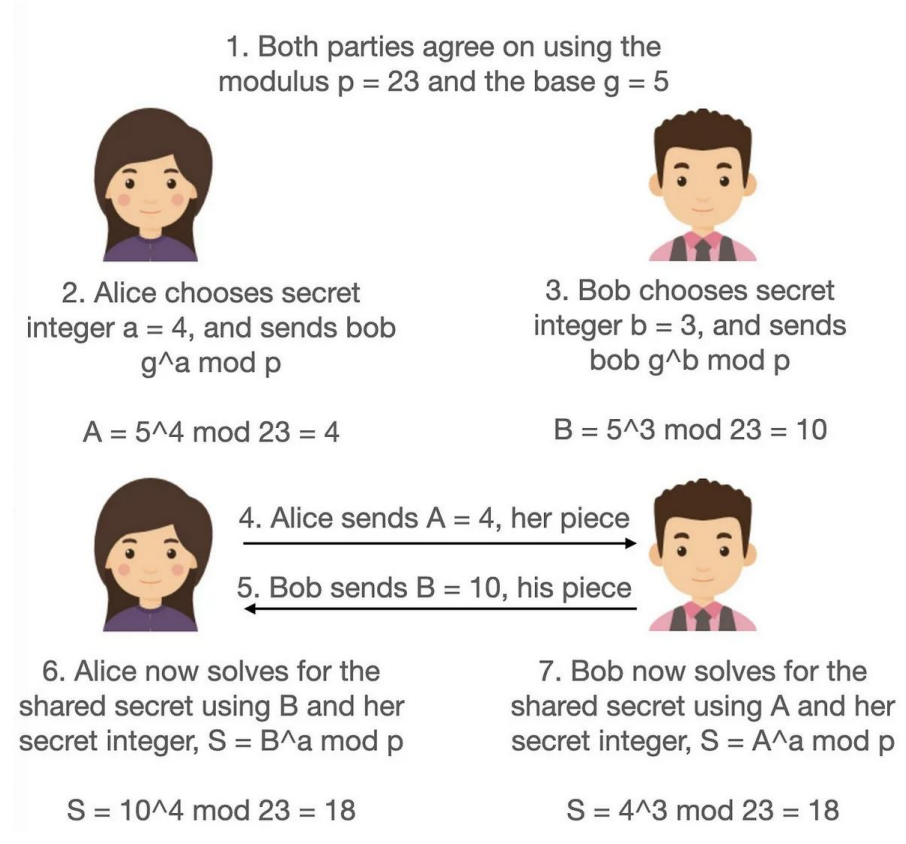
- Alice sends public key A to Bob.
- Bob sends public key B to Alice.

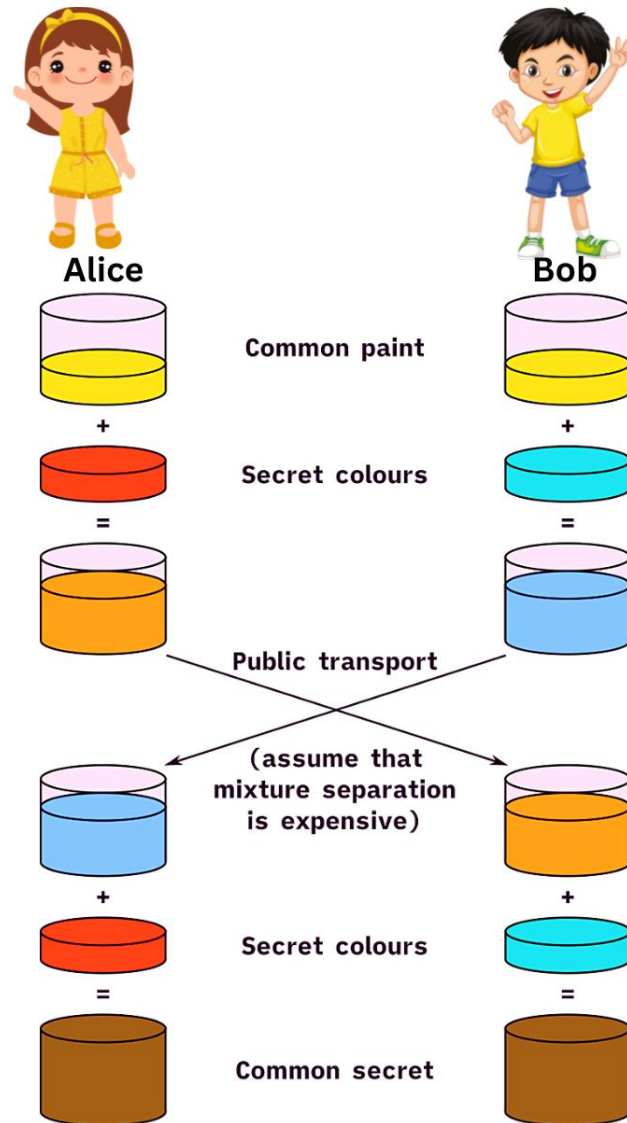
4) Shared Secret Computation

- Alice computes secret: $S = B^a \bmod p$
- Bob computes secret: $S = A^b \bmod p$

5) Result

- Both Alice and Bob end up with the same shared secret S , even though no private data was exchanged directly.

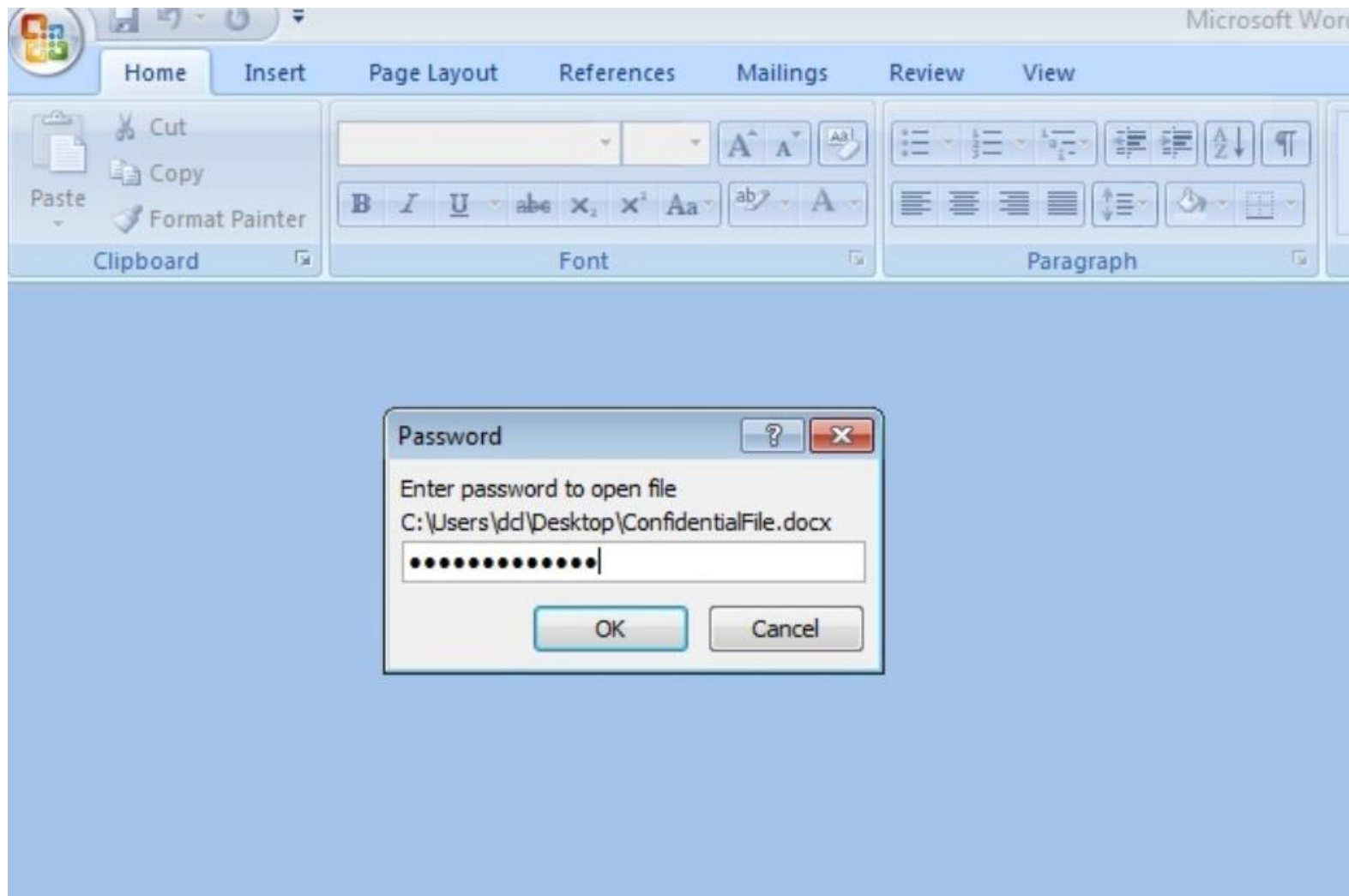


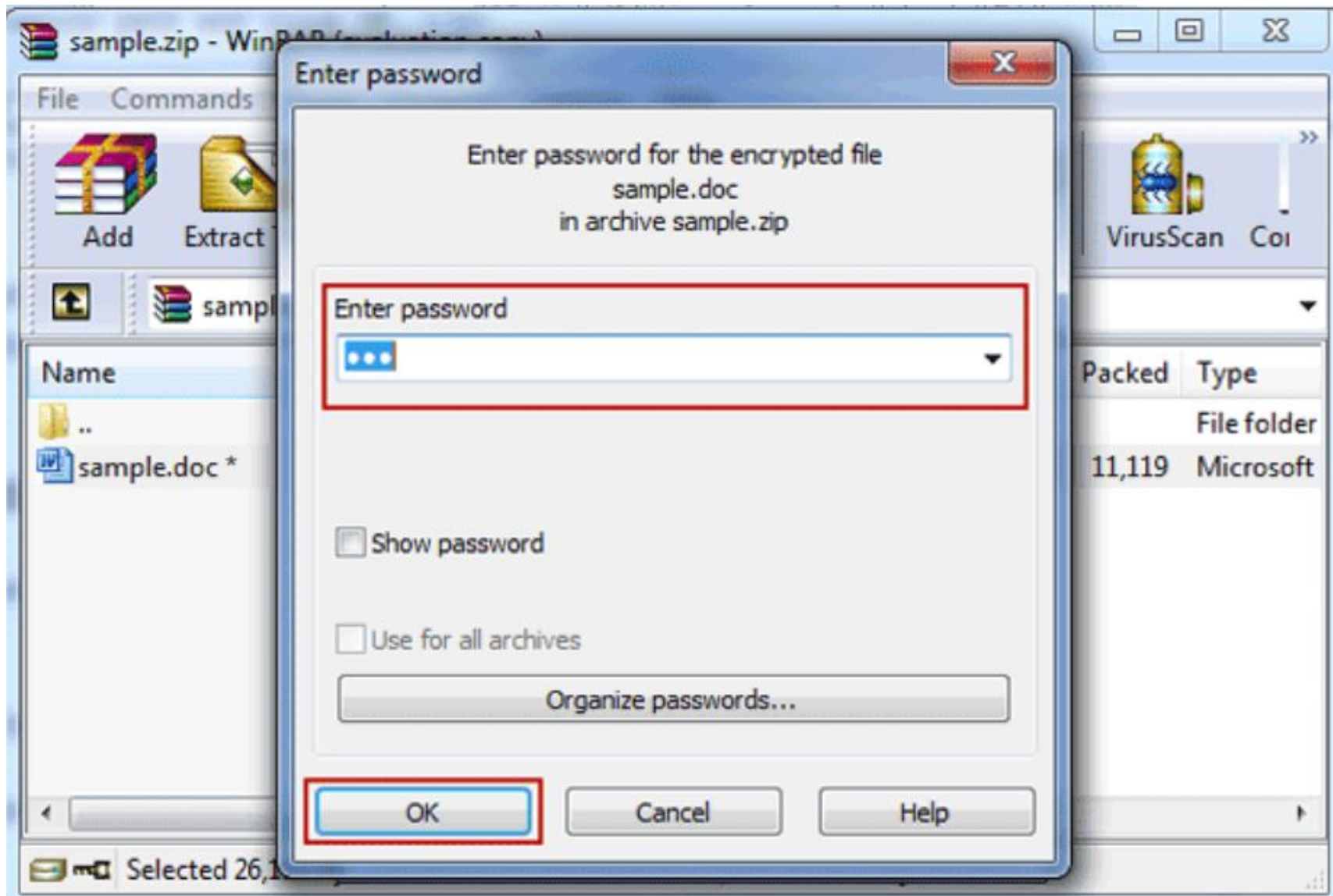


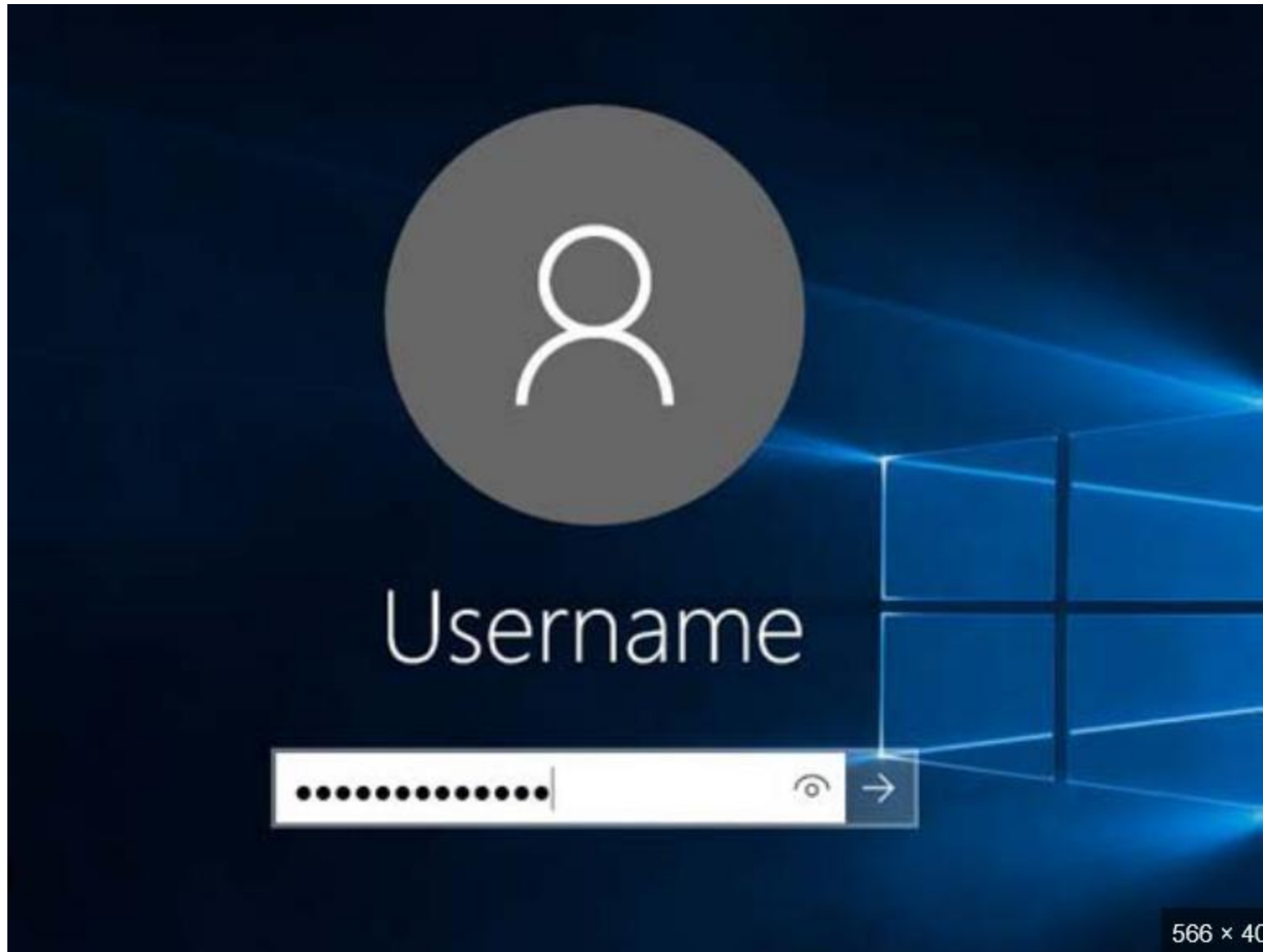
DH Group	Type	Key Length	Description & Security Level	Recommendation
Group 1	MODP	768-bit	Very insecure; easily compromised due to short key length.	Not recommended
Group 2	MODP	1024-bit	No longer secure due to computational advances.	Not recommended
Group 5	MODP	1536-bit	Weaker than current standards; avoid new deployments.	Not recommended
Group 14	MODP	2048-bit	Secure and widely used; safe for current implementations.	Recommended (minimum standard)
Group 15	MODP	3072-bit	Strong security, suitable for sensitive applications.	Recommended
Group 16	MODP	4096-bit	Very strong security, suitable for high-security scenarios.	Recommended for high-security use
Group 18	MODP	8192-bit	Highest security, computationally demanding.	Highly secure, specialized use
Group 19	ECC	256-bit	Modern elliptic-curve; highly secure and efficient.	Highly recommended
Group 20	ECC	384-bit	Strong security, suitable for higher-security needs.	Recommended
Group 21	ECC	521-bit	Very high security ECC, high computational cost.	Highly secure, specialized use

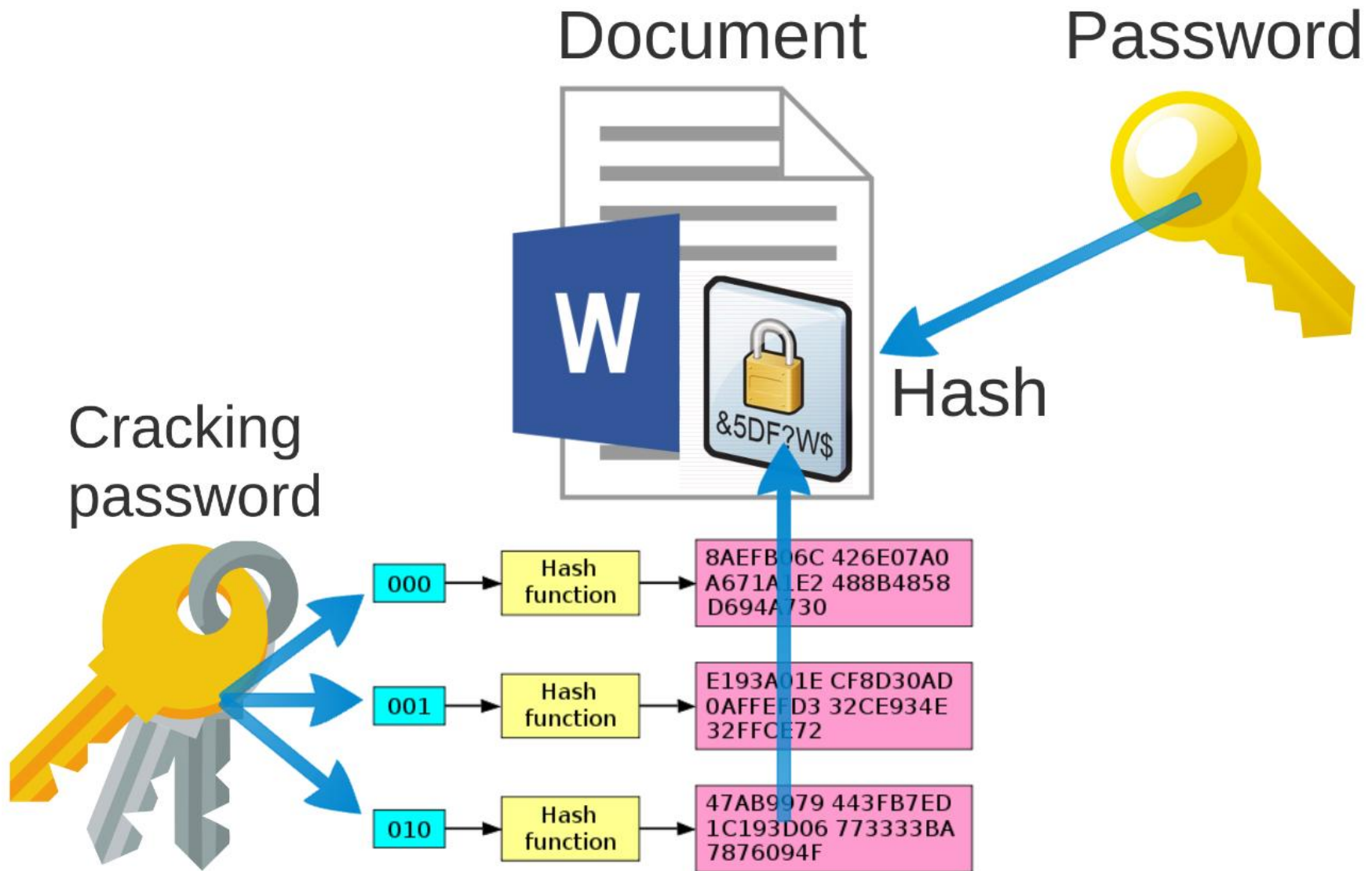
- **DH groups** are predefined sets of parameters used in the Diffie-Hellman key exchange algorithm.
- Each DH group specifies a particular combination of:
 - Prime number (modulus) length (in bits)
 - Generator (base number) for exponentiation
 - Type of elliptic curve (for elliptic-curve groups)
- These parameters **determine the strength and security of the resulting keys** and how resistant the key exchange is against cryptographic attacks.

PASSWORD CRACKING

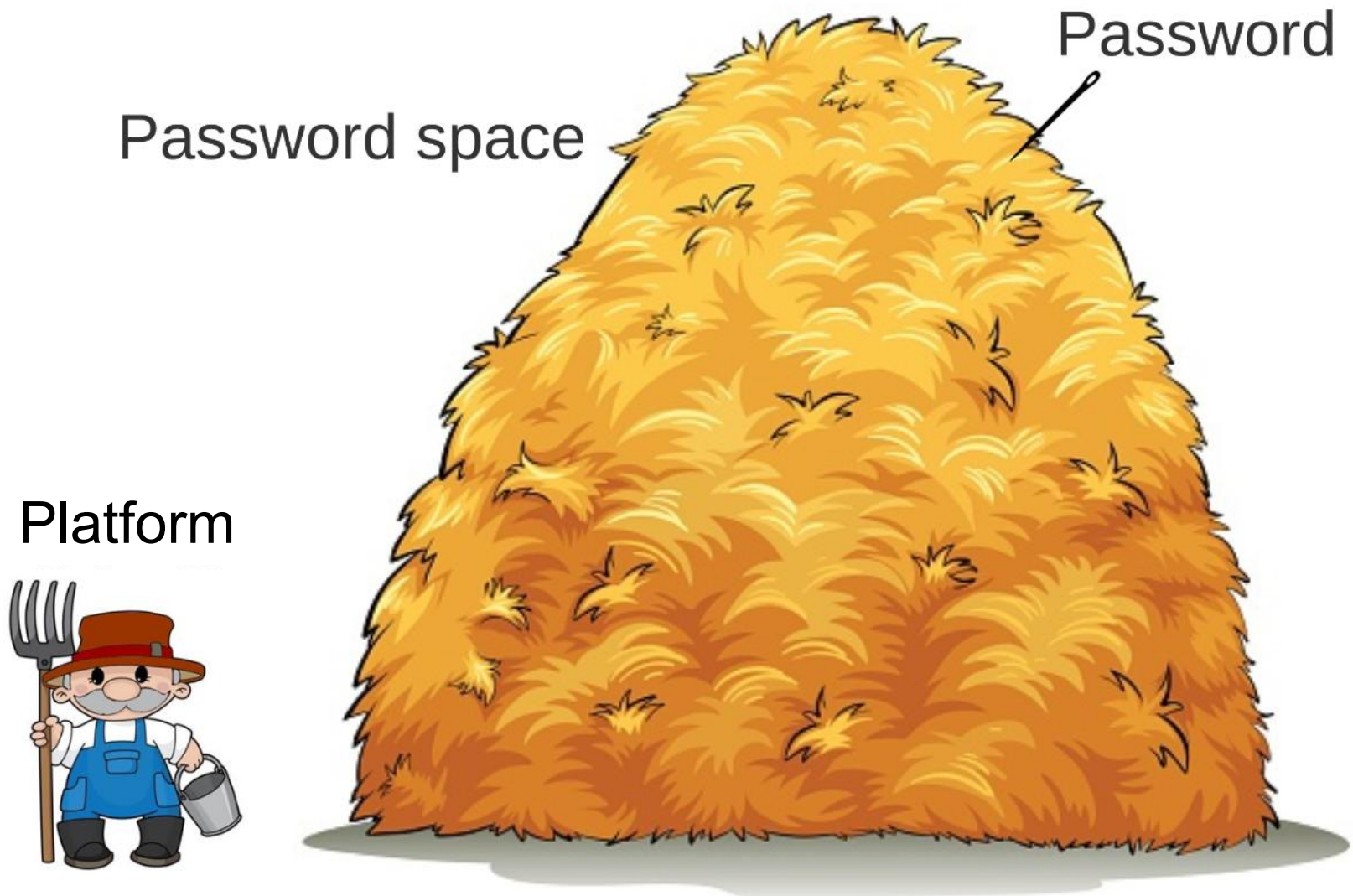










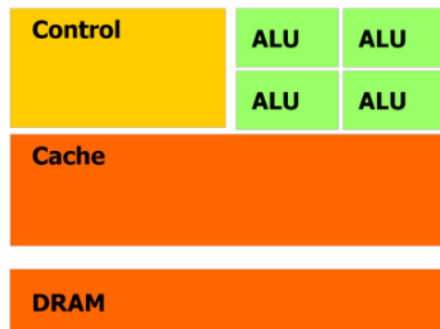


A) What is the password alphabet n ?

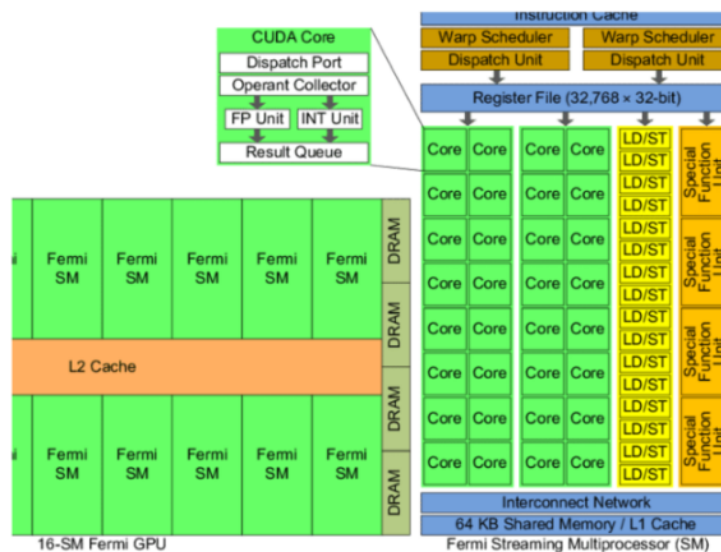
- Numbers = 10
- English alphabet = 26 characters
- Distinguish lower and upper-case = 52 characters
- ASCII = 128 characters
- Russian alphabet = 33 characters
- Japanese primary school Kanji = 2136 characters

B) What is the length k of the password?

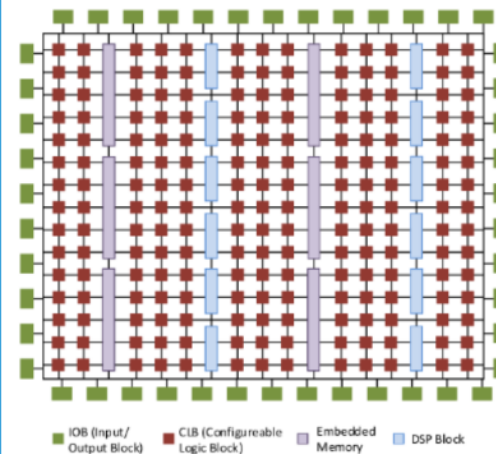
- 1 character long passwords:
a, b, c, d, e, f, ..., x, y, z
- 2 character long passwords
aa, ab, ac, ad, ... zx, zy, zz



CPU



GPU



FPGA



- **Brute-force**

- all candidates,
longest time

- **Dictionary**

- smallest set of
candidates,
shortest time

- **Rule-based**

- subset of candidates,
medium time

Most Common Passwords

1. 123456
2. 123456789
3. 12345678
4. password
5. qwerty
6. 111111

Rotate Left	{	Rotate the word left.	{	p@ssW0rd	@ssW0rdp
Rotate Right	}	Rotate the word right	}	p@ssW0rd	dp@ssW0r
Append Character	\$X	Append character X to end	\$1\$2	p@ssW0rd	p@ssW0rd12
Prepend Character	^X	Prepend character X to front	^2^1	p@ssW0rd	12p@ssW0rd
Truncate left	[	Delete first character	[	p@ssW0rd	@ssW0rd
Truncate right	]	Delete last character	]	p@ssW0rd	p@ssW0r
Delete @ N	DN	Delete character at position N	D3	p@ssW0rd	p@ssW0rd
Extract range	xNM	Extract M characters, starting at position N	x04	p@ssW0rd	p@ss
Omit range	ONM	Delete M characters, starting at position N	O12	p@ssW0rd	psW0rd
Insert @ N	iNX	Insert character X at position N	i4!	p@ssW0rd	p@ss!W0rd
Overwrite N	oNX	Overwrite character at position N with X	o3\$	p@ssW0rd	p@ss\$W0rd
Truncate @ N	'N	Truncate word at position N	'6	p@ssW0rd	p@ssW0
Replace	sXY	Replace all instances of X with Y	ss\$	p@ssW0rd	p@ss\$W0rd
Purge	@X	Purge all instances of X	@s	p@ssW0rd	p@ssW0rd
Duplicate first N	zN	Duplicate first character N times	z2	p@ssW0rd	ppp@ssW0rd
Duplicate last N	ZN	Duplicate last character N times	Z2	p@ssW0rd	p@ssW0rddd
Duplicate all	q	Duplicate every character	q	p@ssW0rd	pp@ssssWW00rrdd

- <https://hashcat.net/hashcat/>
- Open-source password recovery/cracking tool supporting CPU + GPU platforms and all above attacks (and others)
- Supports 300+ hash types (MD5, SHA-256, bcrypt, NTLM, WPA/WPA2, etc.)
- Industry standard for penetration testing & password audits



Q & A

- What are properties of secure communication?
- What is authenticity and how is achieved?
- What is integrity and how is achieved?
- What is confidentiality and how is achieved?
- What is hashing function? Name examples and their properties.
- Describe symmetric-key algorithms. Name examples and their properties.
- Describe asymmetric-key algorithms. Name examples and their properties.

- SCHWARTZ, Stu. *Cryptology for Beginners. Wissahickon HighAmbler, Pa*, 2004, 19002.
- AUMASSON, Jean-Philippe. *Serious cryptography: a practical introduction to modern encryption*. No Starch Press, Inc, 2024.
- MENEZES, Alfred J.; VAN OORSCHOT, Paul C.; VANSTONE, Scott A. *Handbook of applied cryptography*. CRC press, 2018.

Vladimír Veselý veselyv@fit.vut.cz



- <https://www.fit.vut.cz/research/groups/nes@fit/>
- <https://git.fit.vutbr.cz/NESFIT>
- <https://nesfit.github.io>